

ZOROAK

REMOTE SOIL ANALYZER

A. CHEN

R. IJAYA

J. LIANG

C. QIAN

H. TANG

(Received 15 January 2019)

TABLE OF CONTENTS

Table of Contents	1
Zoroak: Circadian Records	3
December 22nd, 2018: Equipment and General Set-up	3
December 24th, 2018: Raspberry Pi Setup and Ultrasonic Testing	5
December 26th, 2018: Servo Motor Testing and Temperature Sensor Testing	7
Servo Testing	11
Further Software Planning	13
December 27th, 2018: Servo Motor Testing and Photocell Testing	15
December 28th, 2018: Arm Prototype and Force Sensor Testing	16
December 29th, 2018: Build Security	19
December 30th, 2018: Building the Base and Camera	21
January 2nd, 2019: Fixing the Base and Imaging	22
Software: Camera Imaging and Object Detection using OpenCV	22
January 3rd, 2019: Camera Live Feed	24
Software: Camera Live Feed + more	25
January 4th, 2019: Stabilizing the Arm and Communication	33
Software: Communication	34
Software Sensors	34
January 5th, 2019: Output	39
Software Output Screen	39
January 6th, 2019: Output	45
Software: Output Display	45
January 7th, 2019: Testing Station	45
Power supply issues	50
January 8th, 2019: Servo Code + Testing Servos on Arm	50
January 9th, 2019: Integration of All Code	59
January 10th, 2019: Installing Soil Testing Station	59
January 12, 2019: Troubleshooting	60
January 13th, 2019: Final Testing	62
January 14th, 2019: Budget	62
Zoroak: Operating Manual	64
SOFTWARE	64
Code Installation	64
Network Configuration	65
User Interface	65
Starting the Arm	65
HARDWARE	66
Arm Setup	66

Testing Station Setup	66
Camera Setup	66
Using the Soil Analyzer	66
Zoroak: Deviations from Prospectus	67
Materials	68
Software	68
Testing Station	68

ZOROAK: CIRCADIAN RECORDS DOCUMENTATION - LOG ENTRIES

TESTING PROCEDURES AND DATA, DAY-BY-DAY SUMMARIES, PROBLEMS AND SOLUTIONS

December 22nd, 2018: Equipment and General Set-up

Balsa wood was bought to construct the arm. Five of the 3x3x12 inch blocks were acquired, however, three more of the 2x2x12 inch blocks were also bought, to see if they were

of more use instead, as the 3x3x12 inch blocks were less wieldy relative to the purchased servos than originally pictured.

Further original musings were effected. The group also worked on the Arduino, code bits were documented for individual components such as sensors or motors.

The first electronic parts were bought. This included a 180 degree servo motor, a micro servo motor, a force sensor, and an ultrasonic sensor.

The specifics of each fundamental constraint was mapped out to further determine the necessary equipment to construct the project, as seen in the prospectus. It was previously determined that for all the mechanical and sensor work, an Arduino would be used to control the arm and the sensors. This was chosen because the Arduino is is more sensitive and has a better control over sensors and movement, compared to a Raspberry Pi. As well, the Arduino would be controlled by a laptop, which would also be connected via bluetooth to the camera. The possibility of using a Raspberry Pi to control the camera and imaging was also considered as it provides greater capabilities and control over a camera module. And thus, a modification with the implementation of a raspberry Pi as the wireless module is shown below, a slight extension to the propositions previously thought up:

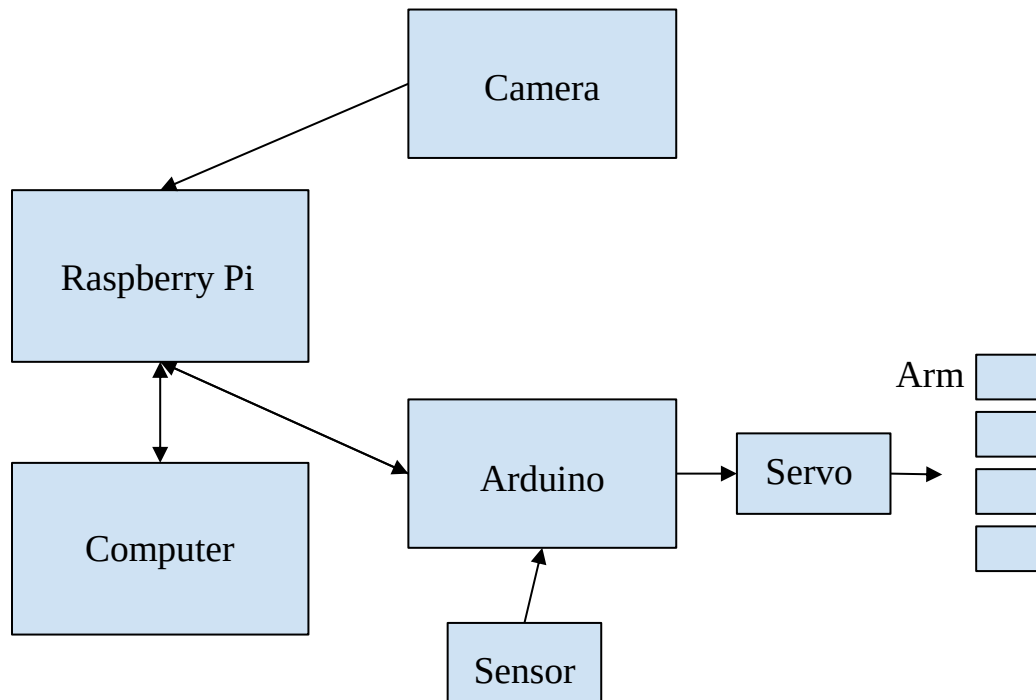


Fig 1. Modifications to the initial proposed bluetooth data processing flow. This was modified further in order to simplify and marginalize possibilities of error in data handling.

It was then suggested that the Raspberry Pi could also act as the microcontroller instead of the Arduino, allowing all sensors and modules to be connected to only the Raspberry Pi via GPIO pins, thereby decreasing the costs and resources. The technology was already readily available.

As well, in order to change, alter, or edit the code for an Arduino, it must be re-downloaded each time, and then run again. Because of the continuous testing and program changes involved in the development stage, this would require a lot time wasted on

downloading the program after each change and is not efficient. Therefore, it was determined that the Raspberry Pi would be the most suitable for this project. The overall flow of the project system was thus further optimized during early stages, due to limiting the number of phases and control modules. However, the core design of the project was not compromised, but rather extended to alternative technologies.

Raspberry Pi and Arduino documentation is already vastly extensive and are made comparable, thus causing no setbacks despite the switch in microcontroller option.

December 24th, 2018: Raspberry Pi Setup and Ultrasonic Testing

A Raspberry Pi 2 was brought in and experimented with. Due to the inaccessible nature of the Arduino in having to re-download the code to edit it each time, it was decided that a Raspberry Pi would be used as the main microcontroller for this project, as per mentioned. Although the Raspberry Pi can act as a computer, all of our data processing will actually occur on an actual computer and the Raspberry Pi will perform the same functions that the Arduino was programmed to do. The Raspberry Pi allowed for more efficiency in the developmental stage and for more coding languages to be used. As a result, the language used would be Python as it is an easier language to work with and supports many libraries that will be useful in the project. Furthermore, the secure shell connection allowed for multiple members to work on the code simultaneously, optimizing group work.

Little progress was made on the first day of actual construction. The majority of the day was spent trying to connect the lab members' laptops to the Raspberry Pi. Upon researching how to do so, a monitor was used to connect the Raspberry Pi to it using an HDMI cable. This was to connect the Raspberry Pi to the internet. However, this seemed to be overly complicated at the time, considering there was no way to type in the internet address onto the monitor, as there was an external keyboard or mouse was not readily available. After some time, a keyboard and mouse was found, and this was done. The second challenge was to connect each laptop to the Raspberry Pi. After much research, one main computer was connected, known as the host. The IP address was also found using the command `sudo config`. Afterwards, each following laptop was connected through Terminal and the IP address. The basic commands used to connect to the Raspberry Pi and host were researched and used. The team discovered how to create new files, how to edit them, and how to run them with code.

Afterwards, research was conducted on how to use various sensors on the Raspberry Pi. Different codes were configured to test each one of the sensors. The temperature sensor and force sensor did not run as expected. Due to the usage of a Raspberry Pi, a slight deviation from the prospected schematics was needed. Utilizing the Pi as a microcontroller also required the purchase of Analog to Digital Converters, due to the lack of built-in functionality. This will be later examined upon further research.

There were numerous errors in the code and setup of the breadboard, many of which were unknown. However, the only sensor in which the code used to run it was working was the ultrasonic sensor. This sensor would be used to measure the distance to the top of the soil, so the height of the soil in the box container is known. This way, the volume can be calculated. The ultrasonic sensor was tested by placing a sheet of wood in front of the sensor, and then slowly moving the wood closer or farther. Each time the wood was moved, the distance from the ultrasonic sensor to the block of wood was measured with a ruler ten times for full uncertainty analysis, and then compared to the value given by the ultrasonic sensor. The

ultrasonic sensor was determined to be accurate and precise within the range of measurements that the group needed (10-20 cm and > 50 cm). The ultrasonic sensor did not detect distances less than 5 cm well, or at all. The minimum distance detected is around 5 cm. This will be kept in mind for future uses and planning.

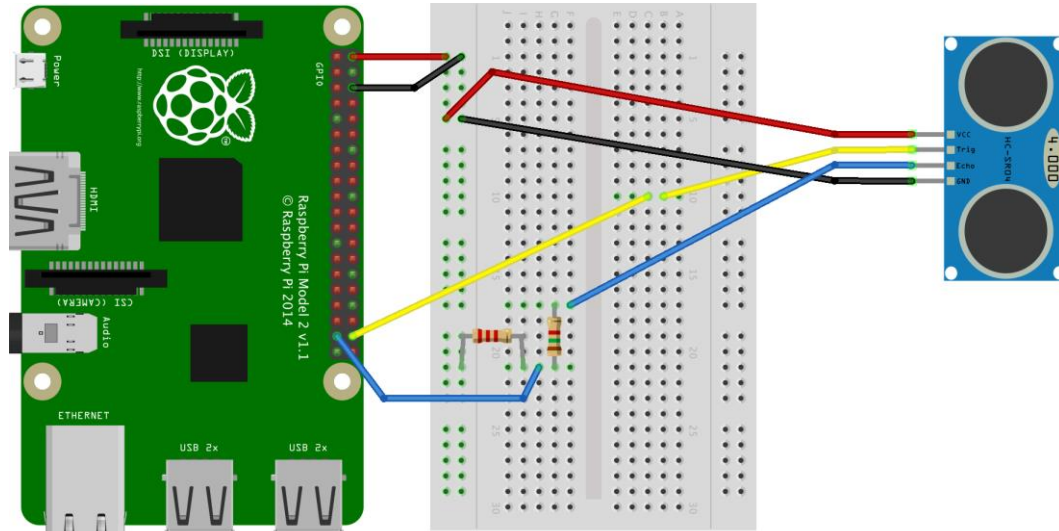


Fig 2. Fritzing diagram of ultrasonic sensor. The relatively simple set-up of a four-pinned ultrasonic allowed for detection of distance via acoustic signals. The values and uncertainty found via experimentation was deemed sufficient for the constraints and purposes of the propositions.

In addition, the group attempted to set up the temperature sensor with the Raspberry Pi, according to the following Fritzing diagram. However, it was determined that the temperature sensor required an analog pin. As the Raspberry Pi has no analog pins and there were no Analog to Digital Converters, the temperature sensor was not set up effectively. After some thought, the group decided to use a digital temperature sensor instead as it can be connected to the Raspberry Pi directly and allowed for the direct display of temperature without have to convert from the potential that was displayed.

This will be further investigated at a later date.

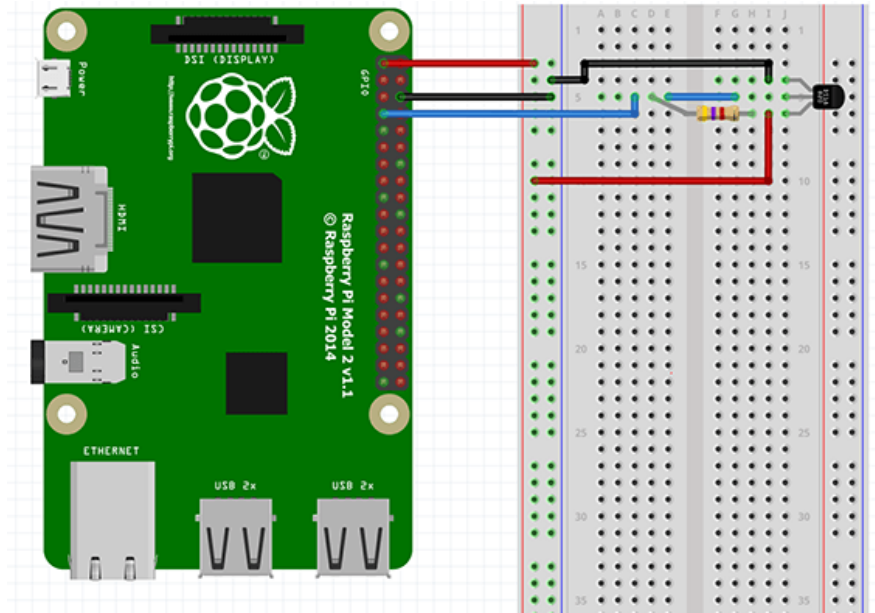


Fig 3. Fritzing diagram of analog temperature sensor. The wiring for future schematics is relatively similar. However, the nature of the output signals of the sensor would heavily improve soil analysis detail.

December 26th, 2018: Servo Motor Testing and Temperature Sensor Testing

More progress was made this day. Firstly, the balsa wood blocks that were intended to be used for the arm were returned. This was mainly because the blocks were deemed too large, even though they were quite light, for the servo motors. It was discovered that the servos are smaller than originally pictured. Significant woodworking would be required for effective placement of the servo motors. It was deemed more effective to veer towards alternative materials discussed prior, whilst still remaining true to the substance of original planning.

Due to manipulation during the experimental phases of the project, the discussed placement of the servos would be unwieldy for possible replacement.

As a result, the group decided to construct the arm out of a found fishing rod. The rod is much thinner, but light and durable, as it needs to withstand the weight of heavy fish. The fishing rod had two detachable parts: an area with the handle which is around 1 metre long, and the area towards the tip that is more flexible. The majority of this part was cut off because it was too bendy and thus wouldn't help the structural integrity of the arm. However, around half a metre was still kept for the second segment of the arm. There are also rings along the rod that will be used to keep the wires detached and organized. The arm is still primarily made of wood.

Although the changing of initial material did occur, the actual design of the prospectus remains constant. The same joints and points of connection will remain fixed, albeit a change of materials due to availability and convenience. The dimensions of the arm were measured a total of ten times and uncertainty analysis was performed. The lengths of the two rods are shown in the table below.

	Length (± 0.05 cm)
--	----------------------------

Long Arm	101.95
Short Arm	42.05

Fig 4. Measurements were taken of the arm. These values are later instilled in the code to precisely move the arm towards designated spot

It was decided that the claw will be made out of an old Coca Cola can. This is to alleviate further stresses on the budget and avoid spending more money.



Fig 5. The bottom half a standard aluminum can was used as the excavator mechanic. The flexibility of the flimsy metal aids in regard to weight and shovelling capability.

A DS18B20 Temperature Sensor was acquired. The digital temperature sensor was able to successfully connect to the Raspberry Pi using the same schematic as the one before (Fig. 3), as it was learned that the wiring between analog and digital temperature sensors were not disparate. The group was able to obtain values that corresponded roughly to the room temperature. The actual testing of the temperature sensor was later roughly conducted, to see if the value given by the sensor matched that of a value given by another thermometer. Further testing was pushed to a later date as calibration would likely change after the sensor would be tampered with during project construction.

Time (s)	Temperature (°C)	Time (s)	Temperature (°C)	Time (s)	Temperature (°C)
1	22.5	55	31.437	108	16.125
2	22.5	56	31.437	109	16.375
3	22.5	57	31.437	110	16.562
4	22.5	58	31.5	111	16.625
5	22.5	59	31.5	112	16.625
6	22.437	60	31.5	113	16.562
7	22.437	61	31.5	114	16.375
8	22.437	62	31.5	115	16.25

9	22.437	63	31.5	116	16.25
10	22.437	64	31.5	117	16.25
11	22.437	65	31.5	118	16.312
12	22.437	66	31.562	119	15.937
13	22.437	67	31.562	120	15.437
14	23.187	68	31.562	121	15
15	25.875	69	31.562	122	14.75
16	27.312	70	31.562	123	14.625
17	28.125	71	31.562	124	14.812
18	28.812	72	31.625	125	14.312
19	29.25	73	31.562	126	13.75
20	29.562	74	31.562	127	13.812
21	29.812	75	31.625	128	13.5
22	29.937	76	31.625	129	13.062
23	30.062	77	31.625	130	12.875
24	30.187	78	31.375	131	12.625
25	30.25	79	30.375	132	12.5
26	30.312	80	28.437	133	12.312

27	30.312	81	26.562	134	12.5
28	30.312	82	25	135	12.75
29	30.375	83	23.562	136	12.687
30	30.437	84	23.125	137	12.75
31	30.437	85	22.187	138	12.812
32	30.437	86	20.687	139	12.875
33	30.5	87	19.625	140	12.875
34	30.5	88	19.062	141	12.812
35	30.562	89	18.812	142	12.75
36	30.562	90	18.562	143	12.937
37	30.625	91	18.125	144	14.25
38	30.687	92	17.687	145	16.25
39	30.75	93	17.312	146	18.812
40	30.812	94	17.062	147	20.187
41	30.812	95	17.25	148	20
42	30.812	96	17	149	19.937
43	30.812	97	16.625	150	19.937
44	30.875	98	16.312	151	19.937

45	30.812	99	16.062	152	19.875
46	30.812	100	15.875	153	19.937
47	31.187	101	15.75	154	19.937
48	31.25	102	15.812	155	20
49	31.312	103	15.562	156	20
50	31.312	104	15.375	157	20.062
51	31.375	105	15.5	158	20.062
52	31.375	106	15.75	159	20.062
53	31.375	107	15.937	160	20.125
54	31.375				

Fig 4.1. Temperature sensor testing data. At 1s, the sensor was exposed to room temperature. At 14s, a hand was placed on the temperature sensor. At 79s, ice was placed on the sensor. At 146s, the ice was taken off and the sensor was once again exposed to room temperature.

Servo Testing

Research was conducted on how to move a servo motor with a Raspberry Pi. The group first connected the servo directly to the Raspberry Pi, based on where each of the three wires were meant to connect to: red to power, black to ground, and orange to the GPIO pin. The group chose 3.3V for this endeavour. However, nothing moved. It was assumed that the power supply was insufficient to move the motor. As a result, it was concluded that an external power source needed to be connected to the circuit. A 9V battery was used for this purpose. As well, because the group wished to maximize the amount of current without exceeding the 12V limit, a transistor was added to the circuit. This was to amplify the current with the power source from the Raspberry Pi as well as that from the battery. The circuit was set up in the following method:

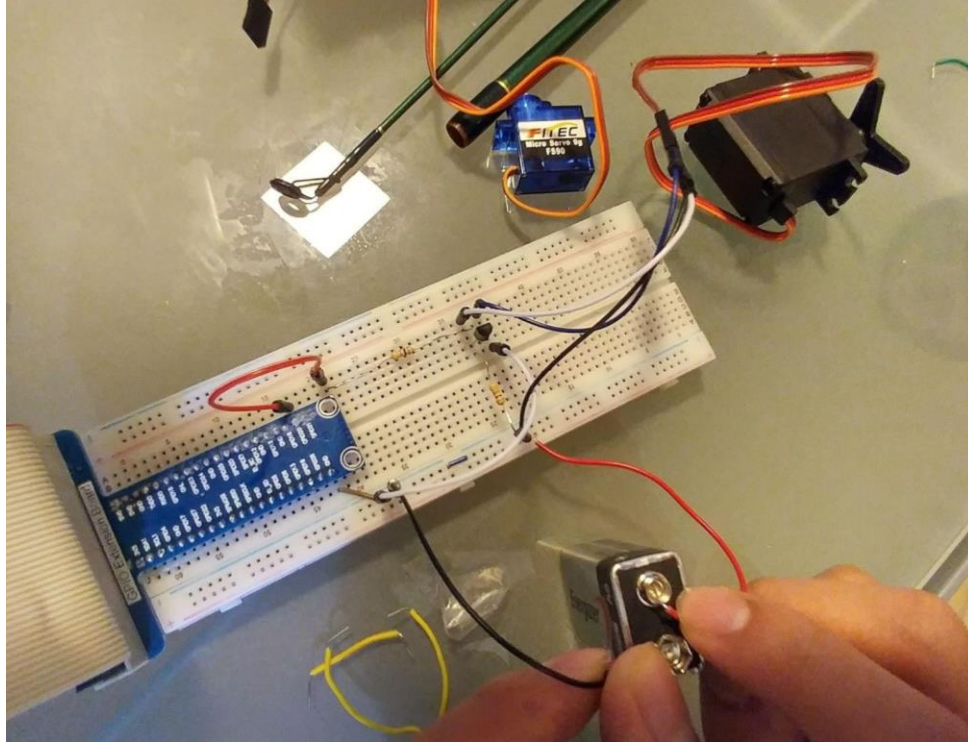


Fig 6. Servo Motor testing phase. A transistor circuit was set-up to amplify the current with limited success. 9 V implementation with NPN transistor.

Servos are controlled by sending an electrical pulse of variable width, or pulse width modulation (PWM), through the control wire. There is a minimum pulse, a maximum pulse, and a repetition rate. A servo motor can usually only turn 90 degrees in either direction for a total of 180 degree movement. The motor's neutral position is defined as the position where the servo has the same amount of potential rotation in the both the clockwise or counter-clockwise direction. The PWM sent to the motor determines position of the shaft, and based on the duration of the pulse sent via the control wire; the rotor will turn to the desired position. The servo motor expects to see a pulse every 20 milliseconds (ms) and the length of the pulse will determine how far the motor turns. For example, a 1.5ms pulse will make the motor turn to the 90-degree position. Shorter than 1.5ms moves it to 0 degrees, and any longer than 1.5ms will turn the servo to 180 degrees. This is all theoretically done, and actually accuracy of the servo motors would be determined by experimental calibration or dynamic adjustments as discussed in the prospectus. The current goal is to achieve general functionality.

A suitable Python code was found to move the motor, that included a transistor. However, there were certain key parts of the code that were currently not understood. For instance, pulse width modulation is the entire theory in which a servo motor moves and is controlled, and this was not fully understood, namely, the following commands:

```
p = GPIO.PWM(11, 50)
p.start(7.5)
p.ChangeDutyCycle(7.5)
```

The numbers inputted above are placeholders. It was discovered that the first number in the bracket of the PWM command indicates the GPIO pin, and the second indicates the frequency. The `p.start` function tells the Raspberry Pi the starting duty cycle. In this case `dc` is the *duty cycle*. The duty cycle describes the proportion of *on* time to the regular interval or *period* of time. If we want a pulse with an specific length we can calculate the duty cycle as follows:

$$dc = \frac{length}{period}$$

Since the group know that the servo uses 20ms cycles, then the three turns of the servo motor can be calculated, as the following:

$$dc = \frac{0.5}{20} \times 100\% = 2.5\%$$

$$dc = \frac{1.5}{20} \times 100\% = 7.5\%$$

$$dc = \frac{2.5}{20} \times 100\% = 12.5\%$$

The final percentage is inputted as a number as the duty cycle.

To change the duty cycle, the command `.ChangeDutyCycle` must be used. It can also be used to change the angle at which the spinning stops. Since the

The servo did not work. There was likely a problem with the code and the PWM commands, as the group was quite confident with the wiring on the breadboard. This problem will be further examined the next day. It is best that finalized wiring of the servo motors are performed with absolute confidence, as without a successful transportation, the soil analysis cannot begin.

Further Software Planning

The group began considering how to implement the software. As the main language on Raspberry Pi is Python, was used to write the programs for the software. In order for the software to have faster processing time and more efficient debugging, the group decided that the software would be divided into smaller sections: Camera Live Streaming, Servo Control and Data Collection, Output Data. In addition, by dividing the software into smaller programs, the group could approach the software in smaller and more manageable pieces. The sections were divided based on which libraries were required for each program and the amount of data shared between the separate programs. A bash script was used to call upon the different programs. The bash script contains a series of linux commands that could run each separate program and pass data between the computer and the Raspberry Pi.

While the overall flow of the program was already pre-planned during the prospectus phase of the project, the introduction of python ssh communication with

a Raspberry Pi adds more room for optimization on the software end of the robotic arm. In the following days, code could be tackled in segmented formatting as planned, in addition to sensor testing via small batches of commands through the raspberry pi interface.

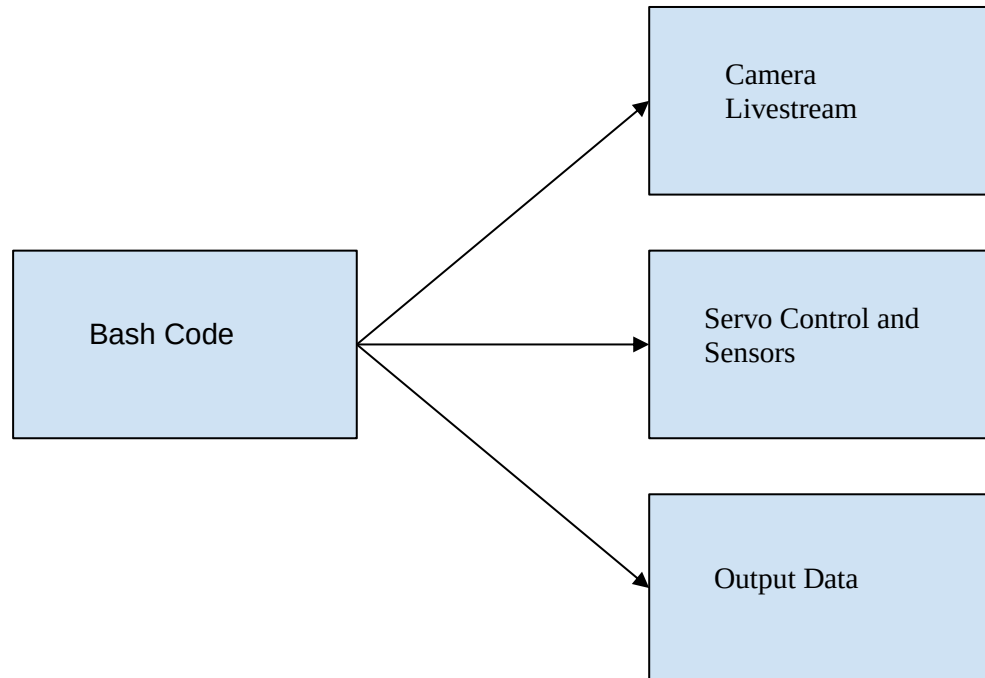


Fig 7. Bash Code flowchart. Executable code from the terminal was used throughout the course of experimentation and the project itself. This was more heavily relied on due to the raspberry pi implementation and thus had to be considered prior to further proceedings.

Upon agreeing to segment the code into separate sections, the group addressed the issues that may occur with this approach. Since the programs (camera, arm, sensor, output, etc) were tackled separately, the group had to figure out an efficient way to communicate between them. After much deliberation, it was decided that file readers and writers would be used to transfer information between the programs. Python offers file reading and writing functions that allows the program to open a file and insert strings into it or open the file and read the lines. With this feature, the codes can receive and give information through text files.

While the livestream camera is running, it continuously writes the time, coordinates, and distance data to a text file. The text file will also be continuously transferred to the Raspberry Pi through the bash script. The program on the Raspberry Pi that controls the arm movement will then extract the data from the text file that has been transferred.

Similarly, with the soil data that is collected by the sensors through the Raspberry Pi, another text file is created and transferred back to the computer. The computer will then be able to run a python program to read in the text file and display the soil sample data onto a separate application.

File reading was thus further researched in subsequent days prior to the implementation of the multi-part program. Keeping previous parts of the software

communication process would likely have more margin for error, alongside more troublesome methodologies for parsing data and producing desired data collection.

December 27th, 2018: Servo Motor Testing and Photocell Testing

The servo motor from the day before was further examined. It was discovered that the duty cycles that were used were inappropriate for the servo, as they were all too high. The group managed to make the servo move to various angles clockwise. The angles were then measured with a protractor to determine if they were accurate or not. Angles less than 170° were quite accurate. However, the servo motor can actually turn a little over 180° , and when the servo motor is told to move to 180° , it overshoots by a quite significant degree. As a result, the group will avoid asking the servo motor to move extremely large angles.

The group also attempted to ask the servo to move counter-clockwise. However, even though the duty cycles were changed to the correct, corresponding ones that would move the servo in the opposite direction, this endeavour was unsuccessful. The goal for the next day will be to have complete control over the servo motor.

The can was cut in a way so that it resembled a claw. Triangles were cut out of the longer edge so that the can could cut through soil and scoop it up. This idea was eventually scrapped as it did not wield any benefits to excavatory mechanics due to the nature of sand. As well, the spinning head of the servo motor was connected to the forearm of the fishing rod.

The group originally had a photocell, and tested it this day. The wiring for a photocell was not simple and the fritzing diagram chosen looked like this:

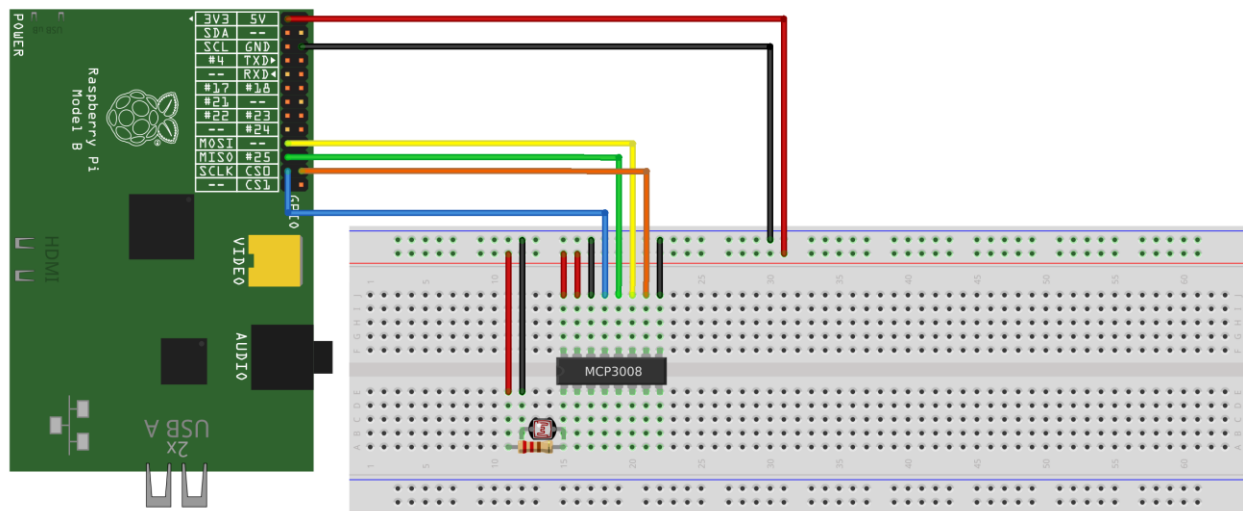


Fig 8. Photocell fritzing chart. Wiring was used during initial musings and projected ideas for obtaining spectra data.

Because Raspberry Pis do not have a built-in analogue to digital converter, the group had to obtain a physical ADC. This would help with the photocell and the force sensor, as that is a analogue sensor as well. Some preliminary research was conducted for the photocell, and it was discovered that the value produced by the photocell is actually a resistance, hence the name photoresistor. However, the relationship between the resistance and the luminosity is not linear. It is more exponential. Therefore, in order to graph the resistance to the luminosity, the luminosity

must also be known, and this is rather difficult. The best way to measure luminosity is through a luminosity sensor, but if that is the case, the group might as well use the luminosity sensor directly

A DS1115 - 16 Bit I2C ADC Module will be obtained sometime soon. Additional components will also be purchased in conjunction with changes made after testing. These slight deviations affect prospectus detailed integrity to some degree, but not the overall flow of the project.

December 28th, 2018: Arm Prototype and Force Sensor Testing

A lot of progress was made this day. The group determined the issue that was causing the servo motor not to move counter-clockwise. Because a transistor was used, the current could only flow in one direction. To move the servo motor in the opposite direction, the current had to flow the opposite direction, and this wasn't possible through the current transistor implementation, if the current was to be amplified. As a result, the transistor was removed and the circuit was rewired according to the following diagram, simply using more voltage to provide more current.

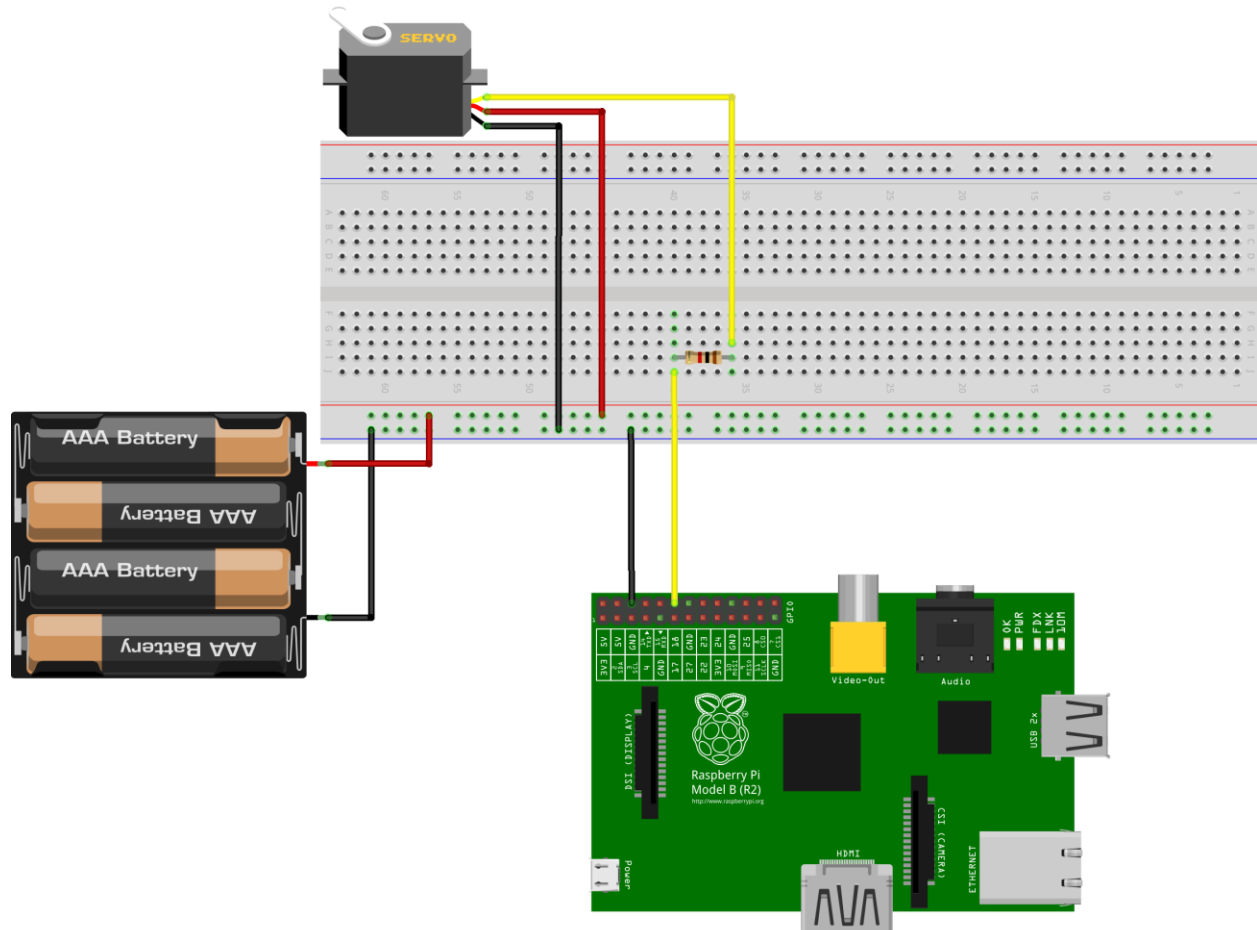


Fig 9. Updated servo diagram. The less involved wiring of the servo turned out to be more effective. This will be repeated several times to complement the planned four servos.

A resistor was added or removed depending on the amount of power required. However, more often than not, it was removed. The group managed to get the servo motor moving counter-

clockwise, and even ask it to move in one direction and switch directions in the middle. Various sample codes were run to test the servo motor.

The group also completed constructing most of the separate parts of the arm. The head attachments of the servo motors were screwed onto the various parts of the arm, which were drilled. The main arm piece was constructed with the bottom of the fishing rod. The group proceeded to make an axle that this arm piece could rotate on. Two blocks of wood were obtained and horizontal holes were drilled so that a thin rod of wood could pass through, being the axle. Straws were strung onto this rod, to lessen the load of friction in the system. The straws were then glued to the inside of the blocks of wood, so that they are secure. All the motors were then attached to the arm. This includes a micro servo for the claw, a 180 degree servo for the forearm, and a continuous servo for the base of the arm. The parts of the arm were all connected together, and the arm was temporarily constructed.

The group made a trip to the electronics store to buy more servo motors and a micro switch, and picked up a box for soil testing and a large plastic plate to be the rotating base, on which the arm will be attached. The axis for the arm was taped down with duct tape, the continuous servo motor was taped down to the back, and temporary chains of jumper wires were connected from the servo motors to the breadboard. The code was then adjusted so that the group could test if each of the servo motors could move and withstand the weight of the arm. The code ran and the arm moved very successfully.

The group also attempted to test the force sensor. It was discovered that the force sensor is an analogue sensor, and in order to use it for the purposes of this soil analyzing design, the sensor needed to produce digital values. An ADC of the type ADS1015 was used for this. The following diagrams provided a reference for the wiring of the sensor (as the ADC types were different, the group had to combine the wiring of the two):

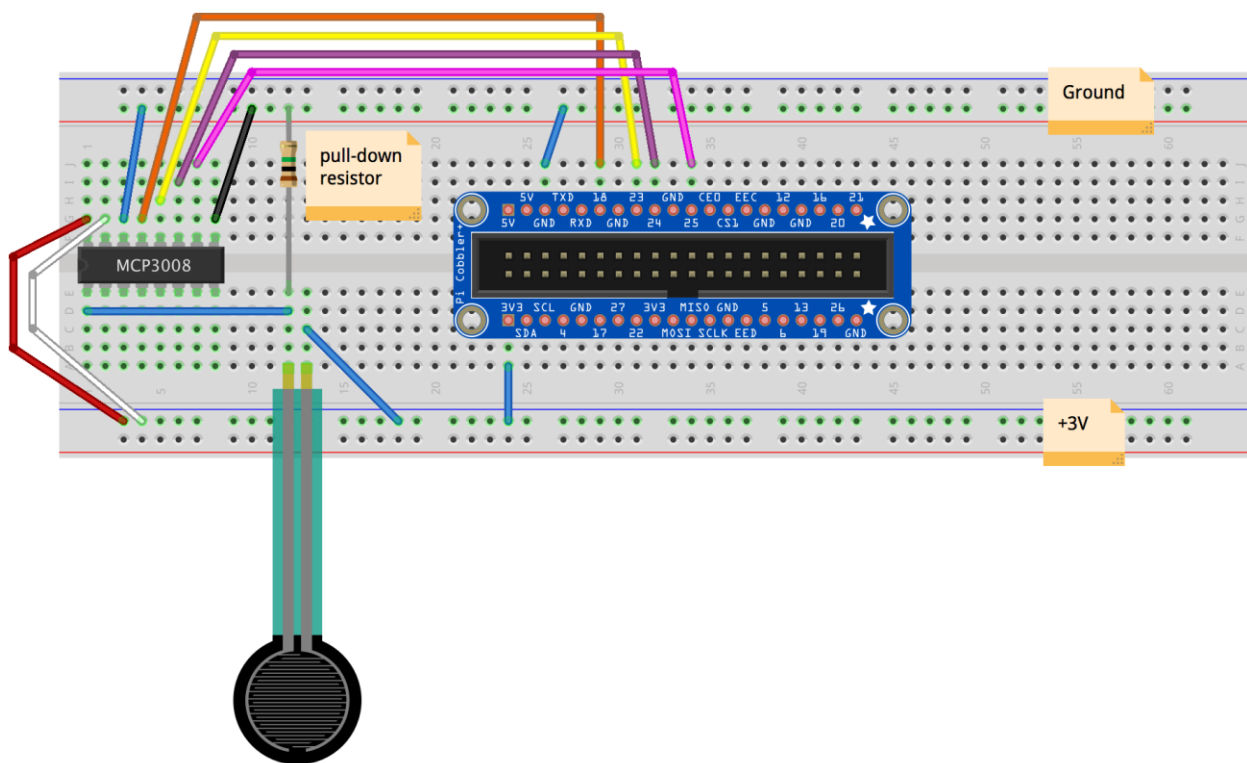


Fig 10. Schematic of force sensor. The force sensitive resistor was connected to an ADC to allow for analog readings and measurements.

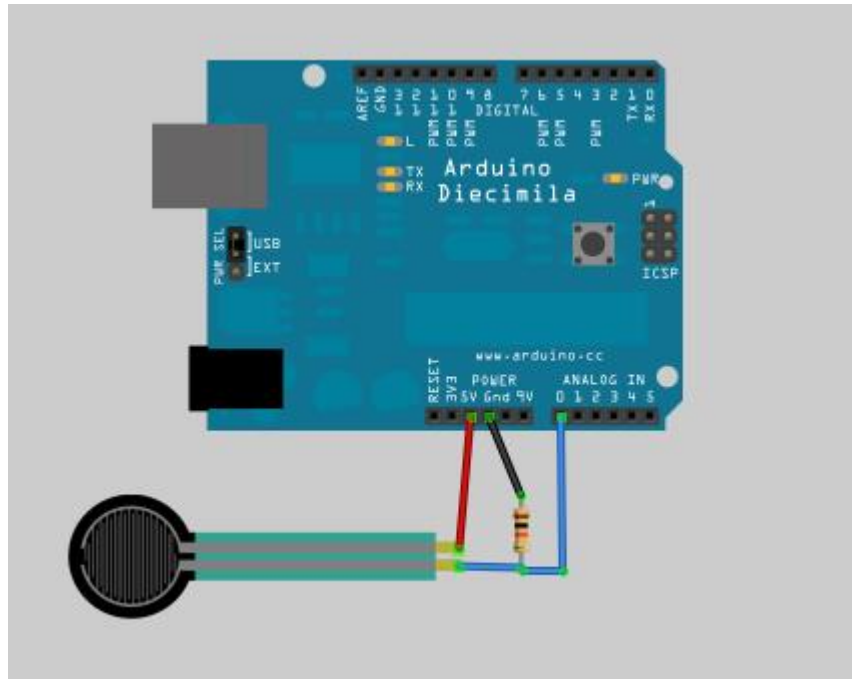


Fig 11. Schematics of force sensitive resistor. The wiring of the force sensor to the ADC mimicked the wiring of the arduino and was used for reference.

The wiring of the ADS1015 ADC was most similar to the wiring of the force sensor to the arduino in that the force sensor would be connected to the power, ground, and one of the four analogue inputs of the ADC. Then the 4 pins of the ADC (ground, power, SCL, and SDA) were attached to the respective pins of the Raspberry Pi. A testing program was run and was able to detect the changes in resistance and potential from the pressure on the force sensor when it was squeezed.

The resistance of the force sensor is usually calibrated to the force using the following graph.

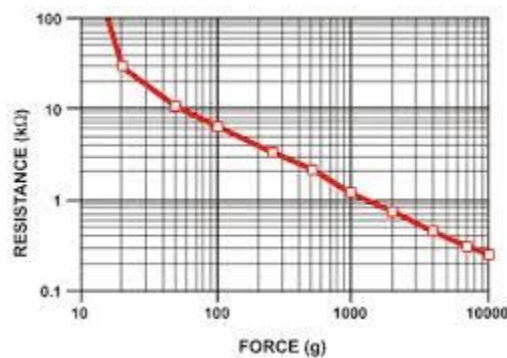


Fig 12. Resistance to Force. A logarithmic graph shows the relationship between the amount of force sense by the force sensitive resistor and the resistance it gives. The graph was used to map values from the resistance to the force.

To simplify the calibration, the group found another resource that had mapped the force to an input potential of 5V. As the group was using a power source that only supplied a maximum potential of 3.3V, some calculations were done to scale the potential.

Force (lb)	Force (N)	Force (kg)	FSR Resistance (ohm)	(FSR + R) ohm	Current through FSR+R (mA)	Potential (5V max) (V)	Potential (3.3V max) (V)
0	0	0.000	Infinite	Infinite	0	0	0.000
0.04	0.2	0.020	30000	40000	0.13	1.3	0.876
0.22	1	0.102	6000	16000	0.31	3.1	2.088
2.2	10	1.020	1000	11000	0.45	4.5	3.031
22	100	10.197	250	10250	0.49	4.9	3.300

Fig 13. Force and resistance. Values from figure 11 were used to find certain points and the relation between sensed force and resistance of the FSR resistor. The values were then regressed to create an exponential curve.

The force values in kg were then graphed against the potential to create an exponential curve. The curve was then regressed and the equation was determined. However, as the force of soil would be really small, it was decided that only a small part of the graph was really needed. So instead of using the exponential function, a series of linear equations were found between each pair of points and the slope was used as the factor to find the force from the potential. This was then coded into the program so that it could be tested.

December 29th, 2018: Build Security

Since the arm can move, the next step is to screw everything down and make sure it is secure. After trying to drill holes into the plastic plate, there were many splits in the plastic, and the plate became slightly deformed. As a result, the plastic plate got scrapped and a square piece of wood was used instead for the base. Holes were drilled for the various pieces of the arm to be screwed down: the servo motor turning the base, the axle holding the arm, and the continuous servo. However, it was noted that a lot of force was needed in order to turn the servo that wound up the string that pulls the arm. Because of this, a pillar was added to make the servo motor higher, so there would be a greater mechanical advantage.

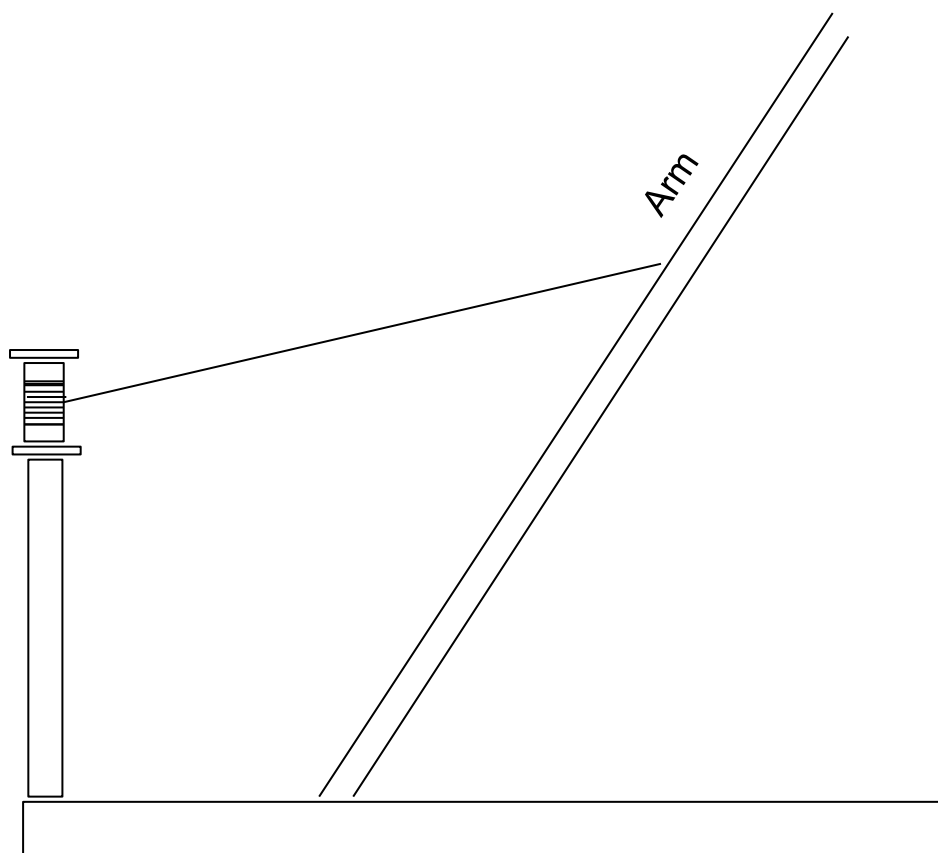


Fig 14. Drawing of Arm. A sketch of the arm was created of the base of the arm. The continuous servo was elevated to provide greater mechanical advantage and to reduce tension in the string

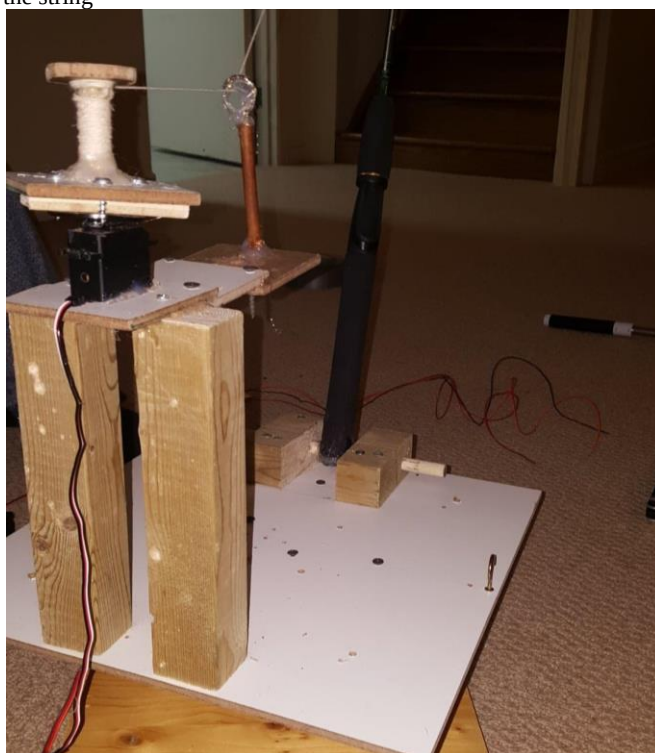


Fig 15. Setup of Base. The base of the soil tester was created based off figure 13. The continuous servo was connected to a spool, which allows it to wind and unwind, which raises or lowers the arm. The servo was elevated by two wooden rods as shown.

December 30th, 2018: Building the Base and Camera

In attempt to stabilize the base, it was decided that a string pulling system would be used to pull the sides of the base taught if it ever tipped, and thus return it to an even position. It was decided that the strings would be tied to hooks, which would all be connected to a single hook at the back of the base, non-rotating. Holes for the hooks were drilled and the hooks were screwed into the base. These hooks were tied very tightly, and they helped a little bit. However, the base was still very unstable because the arm was too heavy, and the connection between the rotating base and the servo motor was very very small.

A spoolie type shape was made for the continuous servo. In order to pull the string that would move the arm up and down, the string had to be connected and wound around something that could easily wind and unwind. For this reason, a spoolie was created out of a dowel and some cardboard at the two ends to prevent the string from escaping the dowel. However, after testing this on the arm, the entire arm failed to move up and down (and crashed very severely) because this cardboard was not sturdy enough to withstand the force from the arm. As a result, more pieces of cardboard were layered on top of each other in attempt to make the structure stronger. It was decided that up to 4 pieces of cardboard was still not strong enough. As a result, relatively thin pieces of wood were used to make the top and bottom of the spoolie. These were heavily glued to the dowel to ensure it does not snap off again, like it did with the cardboard.

A camera stand was created out of wood to allow for a bird's eye view of sampler box. The stand has a total height of around 140cm.

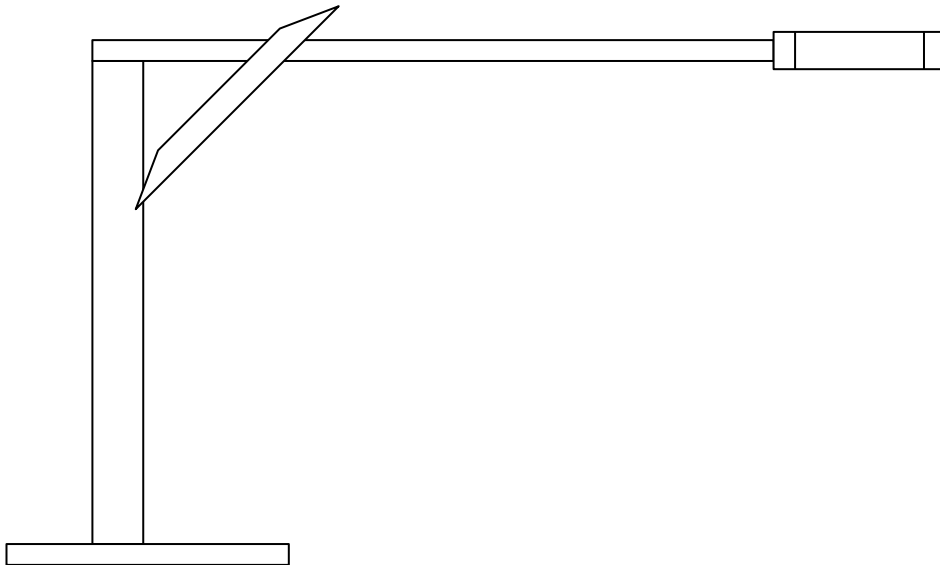


Fig 16. The camera stand. A camera stand was created out of wood. This hovers over the arm to provide a birds eye view of the X-Y plane. A strut was added to provide support for

the horizontal segment and to reduce stress. The stand was installed on a larger base to provide more balance and support.

The camera that will be used is a webcam using the IP Webcam application on a phone. The camera will have a 480x640 pixel resolution and a frame rate of 30 fps. The pixels on the camera were then mapped to the actual distance they covered and the scale factor was determined to be 0.005 m per pixel.

January 2nd, 2019: Fixing the Base and Imaging

More alterations were made to the base of the servo motor. To make the base even more stable, planks of wood were added along the arms of the servo motor to the edges of the rotating base. This would increase the area of the rotating base that the servo motor is directly attached to. However, the pieces of wood chosen were quite thick (around 2 cm). This would add height to the entire rotating plate, and therefore the nail that was originally used to connect the rotating piece to the motor was not long enough. The group attempted to find a screw that is the appropriate height, however this was rather difficult. Holes were screwed and re-screwed multiple times to connect the motor piece to the wooden planks. The motor piece ended up looking similar to completely destructed. Although the group managed to connect all the pieces together, the rotating board was placed on top of the motor and gingerly screwed down with a screw that was too small. A few tests were done by lifting up the arm by the rotating base and seeing if the bottom of the base would fall off. These were all done highly carefully though, as the group was scared that the wood would split and break. However, the screw was truly not screwed in, and the rotating base later became disconnected. This was especially inconvenient because every other piece of the base had to be unscrewed and rescrewed each time this happened, as the screw connecting the plate to the motor had to be screwed in first, before everything else. As well, this was done multiple times as the group had false hope that the screw would fit into the hole. Each time, the base fell apart. It was finally decided that this strategy was not working, and the group decided to remove the planks of wood supporting the arm and figure out a better strategy later.

Software: Camera Imaging and Object Detection using OpenCV

After doing some research, the group found a better solution to a camera module that could save money and time. The group decided on using a phone camera to as the remote webcam that provides live camera/video feed. This would be the most economically feasible choice (it's free). An application, IP Webcam (also free), was downloaded onto the designated phone so that the program could communicate with the application. In order to stream, the program implemented OpenCV that could call upon the phone's ip address to provide live camera feed and display a graphical user interface that would allow the user to click on the place they would like to sample the soil from.

OpenCV is a common infrastructure that is used for computer vision applications and to accelerate the use of machine perception in the commercial products. As an open-source library that is compatible with Python, it allows for real-time computer vision. The library has many capabilities, such as image detection through Hughes lines and image kernels, as well as machine learning implementation through neural networks.

To test the live camera feed, the program was first connected to the computer's webcam, using a template was taken from the Python OpenCV library. After making sure it worked, the camera feed was then initialized as the webcam on the the phone. The live camera/video feed will continue to run until a key is pressed to exit the loop. Upon exiting the loop, the program will close.

```
-----
# import
import cv2
import numpy as np
import urllib
import time
import argparse

circles = []

#Define mouse click event
def click(event, x, y, flags, param):
    global xcoord, ycoord

    # Mouse click
    if event == cv2.EVENT_LBUTTONDOWN:
        #get x,y coordinates
        xcoord = x
        ycoord = y
        print (xcoord, ycoord)
        circles.append((x,y))\

url='http://192.168.2.50:8080/'
stream = cv2.VideoCapture(url+'video')

i = 1
ret, img = stream.read()
while(True):
    ret, frame = stream.read()

    #img = cv2.imread('img.png',0)
    # Stream live video
    cv2.imshow('My Live Stream', frame)

    # Still image on 10th frame
    if i == 10:
        img = frame
        if(i%15)==0:
            #cv2.imshow('image',frame)
```



```

cv2.setMouseCallback("image", click)

for center_position in circles:
    cv2.circle(img, center_position, 5, (0,0,255),
-1)

cv2.circle(img, (30,30), 5, (0,0,255), -1)

cv2.imshow('image',img)]

i+=1

key = cv2.waitKey(1)
if key & 0xFF == ord('q'):
    break
elif key == ord('d'):
    circles = []

cv2.destroyAllWindows()
-----

```

An example window would be:

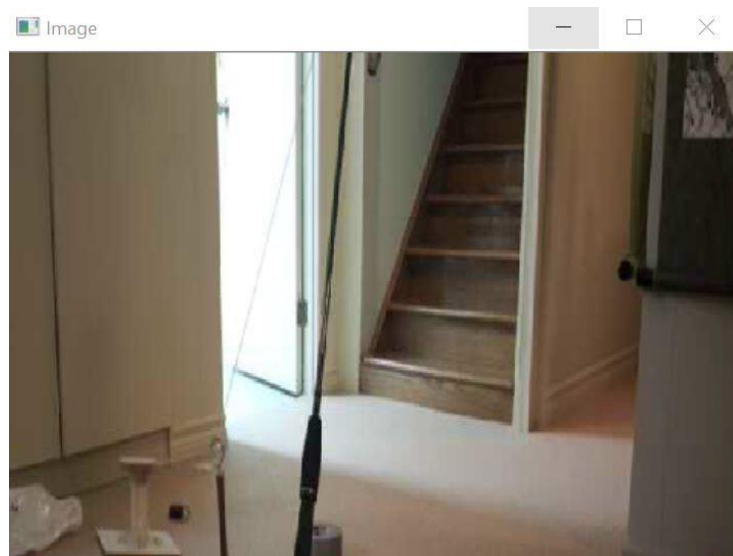


Fig 17. OpenCV software example. OpenCV was used to provide live camera feed. This is an example of a python program involving opencv, which is providing real time computer vision.

January 3rd, 2019: Camera Live Feed

The pieces that still needed to be screwed in and secured were done so. Many pieces that needed to be screwed down were done so. This took longer than expected mainly because the organization of the parts sometimes changed slightly, and sometimes measurements were not exact, and affected the placement of various parts of the arm.

Software: Camera Live Feed + more

The camera live feed program was modified to implement more features, which will enhance the user experience. The program saves and displays the 50th frame as a still image and a mouse listener is added to the image to allow for user input. Mouse events notify when the user uses the mouse to interact with a component, which occurs when the cursor enters or exits a component's onscreen area or when the user presses or releases one of the mouse buttons. This program implements a listener for when the left mouse button is clicked. The program lists down the coordinates of the mouse when the left cursor is pressed down. A dot would appear on the coordinates which indicate where the user wants the arm to go. The user is also allowed to delete dots if they choose not to go to that point and to save their choice by pressing different keys. When the "s" key is pressed, the coordinates are saved and the program then calculates the angle and distance it is from the arm and writes them to a file.

Experimentation was performed to create a colour-based tracking system to determine the location of the arm in the livestream. Using the functions offered by the OpenCV library, a coloured mask was placed on top of the livestream, masking everything except a specific range of hues. A blue ball was chosen to be placed on the arm as it was the most contrasting colour from the surroundings and its shape allowed for the same view from all angles.

Using a similar concept, live depth detection was also coded into the program using the focal length of the camera.

Every 20 frames after the user saves the coordinates, the angle, distance, and height are calculated and written into a file.

Additional features include:

Features	Description
Depth Perception	Using the Opencv library, the code is able to provide real-time depth perception
Arm detection	The OpenCV library will also be able to detect the position of the arm using a colour mask that tracks a specific colour through the whole live stream. A blue ball was placed on the arm which would contrast the primarily red/yellow-ish background of the soil, allowing for a more accurate detection.
User input	A mouse key listener will be used to get user input from the user.
Soil Sample Location	The user is able to click the position on the screen to indicate the location of the soil to sample. This provides coordinates on the screen which will later be translated into real time coordinates. The GUI will display this location on the point where the use clicked
Base Location	The user will also click the position of the base of the soil sampler arm. The coordinates will provide a reference point of where the arm will move from to the soil sample location coordinates. The GUI will display this location on the point where the use clicked
Calculations	With the coordinates obtained from the soil sample location and base location, the distance and angle of the soil sample and arm from the base was calculated.

	These polar coordinates will be written onto the web server through JSON in order for it to be accessible to the next program (Servo arm)
Sample Picture	Since the program requires a picture of the soil sample, OpenCV is used to crop the image from the live stream video. The coordinates of the soil sample location is used as a reference to the location at which the picture will be cropped as a square of size 100x100 px. The image is then concatenated with a color reference strip to allow for color reference.

Fig 18. Camera feed program features. The code will have the following features: depth perception, arm detection, and user inputs like soil sample location, base location. These will be used for calculations that will be used in the next program which provides servo control. At this stage, a sample picture of the soil sample is taken and cropped. This is used to identify the soil sample which will be displayed in the program that displays output values.

The application will look like the following:

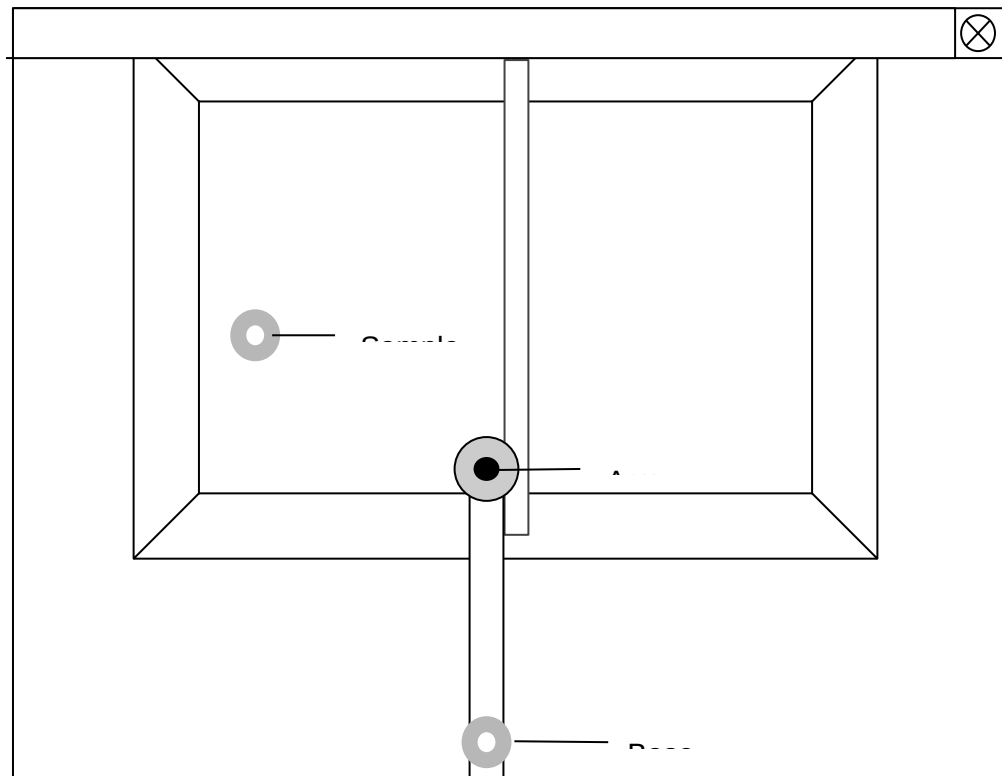


Fig 19. Window of camera feed. The window for the live camera feed is designed to show a birds-eye view of the plane of where the arm will operate. Labels are used to identify the locations of the base and testing sample, which are inputted by the user. The arm is indicated by the blue ball installed on the arm, which is detected using the OpenCV library.

The image of the soil sample will look as the following:

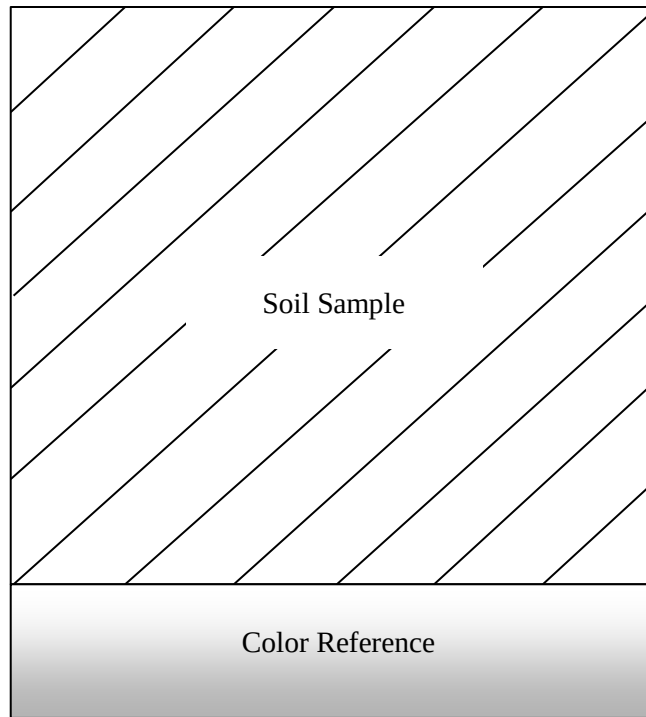


Fig 20. Soil Sample Image, This is the format of the soil sample image which will be used later on in the output program.

The following is the code:

```
-----

#!/usr/bin/env python

# import
from sys import stdin, stdout
from imutils.video import VideoStream
from imutils import paths
import imutils
import cv2
import numpy as np
import urllib
import time
from math import *

# Declare variables
## ALL MEASUREMENTS ARE IN METRES

url = 'http://192.168.2.50:8080/'
base = False
start = False
font = cv2.FONT_HERSHEY_SIMPLEX
factor = 0.003270641 # 1 pixel = 0.18 cm
```

```

# Colour mask
blueUpper = (110, 255, 255)
blueLower = (90, 115, 115)

orgDist = 1 # This value actually matters

# Initialize values (the initial values don't actually
matter)
center = (0, 0)
radius = 1
orgRad = 1
focLength = 500
height = orgDist

# Define mouse click event
def click(event, x, y, flags, param):
    global xcoord, ycoord
    global circen

    # Mouse click
    if event == cv2.EVENT_LBUTTONDOWN:
        # get x,y coordinates
        xcoord = x
        ycoord = y

        cv2.circle(img, (xcoord,ycoord), 10, (0, 0, 255), -
1)

        cv2.imshow('Image', img)

def measure(x, y, armx, army, h):
    global angle, distance

    # Determine relative x+y distance
    xrel = (x - xBase) * factor
    yrel = (y - yBase) * -1 * factor

    # Determine relative arm x+y distance
    xarel = (armx - xBase) * factor
    yarel = (army - yBase) * -1 * factor

    # Determine angle and distance of coordinate
    if xrel != 0:
        angPt = degrees(atan(yrel / xrel))

```

```

        if angPt < 0:
            angPt += 180
    else:
        angPt = 90

    # Determine angle and distance of ball
    if xarel != 0:
        angBall = degrees(atan(yarel / xarel))
        if angBall < 0:
            angBall += 180
    else:
        angBall = 90

    # Determine distance of coord and ball
    distPt = sqrt(pow(xrel, 2) + pow(yrel, 2))
    distBall = sqrt(pow(xarel, 2) + pow(yarel, 2))

    # Displays values and exports them into a text file
    mystr = ("%3f %3f %3f %3f %3f %3f %3f" % (xrel,
yrel, angPt, distPt, angBall, distBall, h))

    f = open("values.txt", "a+")
    f.write(mystr + '\n')
    f.close()

#### MAIN #####

# Creates Windows
window = cv2.namedWindow('Image', cv2.WINDOW_NORMAL)
cv2.resizeWindow('Image', 480, 640)
cv2.moveWindow('Image', 500, 50)
cv2.setMouseCallback('Image', click)
window2 = cv2.namedWindow('Live Stream', cv2.WINDOW_NORMAL)
cv2.resizeWindow('Live Stream', 480, 640)
cv2.moveWindow('Live Stream', 10, 50)

# Creates stream
stream = cv2.VideoCapture(url + 'video')

# Start stream
i = 1
while (True):
    ret, frame = stream.read()
    cv2.putText(frame, 'Press \'q\' to exit.', (10, 520),
font, 0.7, (255, 255, 255), 1, cv2.LINE_AA)

```

```

# Still image on 50th frame
if i == 50:
    img = frame
    cv2.imwrite("img.png", img)
    cv2.putText(img, 'Click on where you want to go.',
(10, 500), font, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
    cv2.putText(img, 'Press \'d\' to delete the point.',
(10, 540), font, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
    cv2.putText(img, 'Press \'b\' to select the base.',
(10, 560), font, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
    cv2.putText(img, 'Press \'s\' to select the soil
sample.', (10, 580), font, 0.7, (255, 255, 255), 1, cv2.LINE_AA)
    cv2.imwrite("stillframe.png",img)
    cv2.imshow('Image', img)

# Display point on livestream feed
if base == True:
    cv2.circle(frame, (xBase,yBase), 10, (0, 0, 255), -
1)
    cv2.putText(frame, 'Base', (xBase+10, yBase+10),
font, 0.5, (255, 255, 255), 1, cv2.LINE_AA)
    if start == True:
        cv2.circle(frame, (xsoil,ysoil), 10, (0, 0, 255), -
1)
        cv2.putText(frame, 'Soil Sample', (xsoil+10,
ysoil+10), font, 0.5, (255, 255, 255), 1, cv2.LINE_AA)

# Colour Motion Detector
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# construct a mask for the color "green", then perform
# a series of dilations and erosions to remove any small
# blobs left in the mask
mask = cv2.inRange(hsv, blueLower, blueUpper)
mask = cv2.erode(mask, None, iterations=2)
mask = cv2.dilate(mask, None, iterations=2)

# displays mask of blue colour

# find contours in the mask and initialize the current
# (x, y) center of the ball

cnts=cv2.findContours(mask.copy(),cv2.RETR_EXTERNAL,cv2.CHAIN
_APPROX_SIMPLE)
cnts = imutils.grab_contours(cnts)

```

```

        cnts = sorted(cnts, key=cv2.contourArea,
reverse=True)[:3]
        # Sort 3 largest contours

        # only proceed if at least one contour was found
        for c in cnts:
            peri = cv2.arcLength(c, True)
            approx = cv2.approxPolyDP(c, 0.015 * peri, True)

            # only proceed if there are not too many edge points
            (x, y, w, h) = cv2.boundingRect(approx)
            ar = w / float(h)

            if ar >= 0.8 and ar <= 1.2:
                # find the largest contour in the mask, then use
                # it to compute the minimum enclosing circle and
                # centroid
                ((x, y), radius) = cv2.minEnclosingCircle(c)
                M = cv2.moments(c)
                center = (int(M["m10"] / M["m00"]), int(M["m01"]
/ M["m00"]))

                # draw the circle and centroid on the frame,
                # then update the list of tracked points
                cv2.circle(frame, center, int(radius), (0, 255,
255), 2)

                cv2.circle(frame, center, 5, (0, 0, 255), -1)
                cv2.putText(frame, 'Arm', (center[0]+10,
center[1]+10), font, 0.5, (255, 255, 255), 1, cv2.LINE_AA)
                # Displays coordinates every 20th
                frame

                if start == True and i % 20 == 0:
                    measure(xsoil, ysoil, int(x), int(y),
height)

                break

        # Initialize initial focal length
        if i == 5:
            focLength = (radius * orgDist) / orgRad

        # Calculates height
        elif i >= 5 and i % 20 == 0:
            height = orgRad * focLength / radius

        # Displays height
        if i >= 5:

```



```

        cv2.putText(frame, "%.02fm" % height,
                    (frame.shape[1] - 250, frame.shape[0] -
20), cv2.FONT_HERSHEY_PLAIN,
                    2.0, (0, 255, 0), 3)

    # Stream live video
    cv2.imshow('Live Stream', frame)

    # Counter
    i += 1

    # Break out of loop
    key = cv2.waitKey(1)
    if key & 0xFF == ord('q'):
        break

    elif key == ord('d') and start == False:
        point = False
        circen = []
        img = cv2.imread('stillframe.png',-1)
        if base == True:
            cv2.circle(img, (xBase,yBase), 10, (0, 0, 255),
-1)
            cv2.putText(img, 'Base', (xBase+10, yBase+10),
font, 0.5, (255, 255, 255), 1, cv2.LINE_AA)
            cv2.imshow('Image',img)

        elif key == ord('b') and base == False:
            xBase = xcoord
            yBase = ycoord
            cv2.putText(img, 'Base', (xBase+10, yBase+10), font,
0.5, (255, 255, 255), 1, cv2.LINE_AA)
            cv2.imshow('Image',img)
            base = True

        elif (key == ord('s') and start == False and base ==
True):
            try:
                xsoil = xcoord
                ysoil = ycoord
                cv2.putText(img, 'Soil Sample', (xsoil+10,
ysoil+10), font, 0.5, (255, 255, 255), 1, cv2.LINE_AA)
                cv2.imshow('Image',img)

                # Call measure function
                measure(xsoil, ysoil, xBase, yBase, height)
                start = True

```

```

stfile = open("start.txt", "w+")
stfile.write("1")
stfile.close()

# Save soil image with colour reference
soil = cv2.imread("stillframe.png",-1)
cropped = soil[ycoord-50:ycoord+50, xcoord-
50:xcoord+50]
colour= cv2.imread("ColourReferenceScale.jpg",-
1)

r = 100.0 / colour.shape[1]
dim = (100, int(colour.shape[0] * r))
resized = cv2.resize(colour, dim, interpolation
= cv2.INTER_AREA)
vertical = np.concatenate((cropped, resized),
axis=0)

cv2.imwrite("SoilColour.jpg",vertical)
except:
    # If no coordinate points were initialized
    print("No coordinate point was specified.")

cv2.destroyAllWindows()
-----

```

A screenshot of the application is shown below:

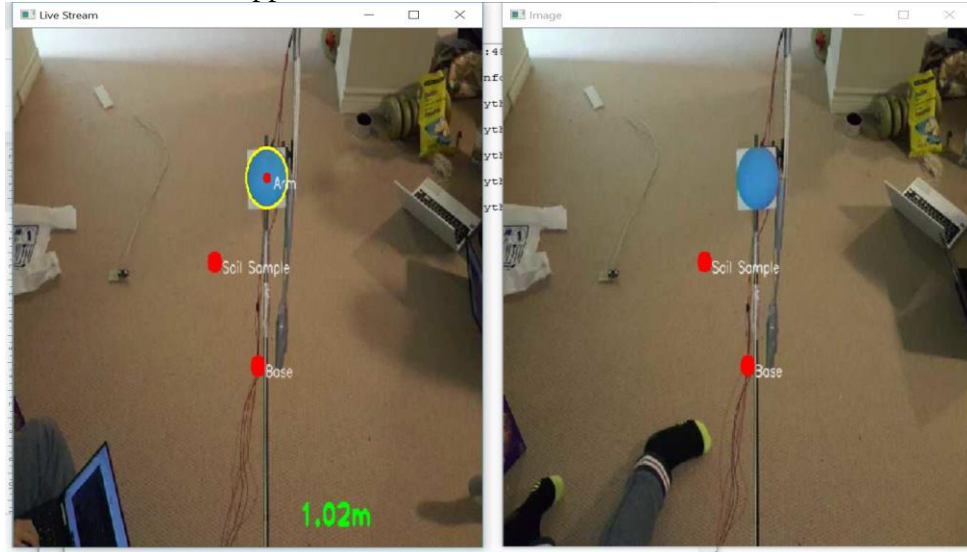


Fig 21. Screenshot of camera feed application. The program runs. Two windows are shown; the right one is a still image of a frame taken 0.1 seconds after the program begins and the left one is a live stream. The live stream shows depth perception as well as tracks the movement of the arm.

January 4th, 2019: Stabilizing the Arm and Communication

A better idea to stabilize the base of the arm was decided upon. It was suggested that three poles/dowels could be attached on the rotating base, each the exact length between the rotating base and the stationary one. This way, when the centre of mass moves forward or backwards depending on whether the arm moves forward or backwards, the weight added to the front of the base will not tip the front, as the dowel will resist the pressure. The exact height of each dowel was difficult to mimic. By accident, the group made two of the three dowels shorter than intended. This was fixed by adding folded cardstock to the bottom of the poles, to increase the height. These poles were then drilled into the board, and the base was tested for tipping. There was zero tipping, and very minimal wobbling. Because of the success of this addition/idea, the strings attached to hooks idea was scrapped, but the extended planks at the bottom of the board was kept.

Software: Communication

It was later determined that the file transfer from a data file was too slow for live correction of the arm movement. Instead, another python script was written to convert the data into its representative JSON format and displaying it on a webpage. The Raspberry Pi connected to the same network would be able to retrieve this data from the webpage and parse the JSON format back into the data variables that are used by the arm.

Software Sensors

With the data used to measure the readings through the sensor, the code for each sensor was combined to create the program for the testing data. Each sensor will collect data for a total of 10 times (except for the luminosity sensor which will only collect 6 data points for the reflective spectra) and the results will be averaged. This will allow for more noise cancellation and more precise readings.

The following shows the code that was used for the sensors:

```
#Libraries
import RPi.GPIO as GPIO
import time
import smbus
import Adafruit_ADS1x15
import math
import random

# LED Variables
wavelength = [700, 610, 570, 495, 420, 380]
R = [255, 255, 255, 0, 0, 128]
G = [0, 165, 255, 255, 0, 0]
B = [0, 0, 0, 0, 255, 128]

# Create an ADS1015 ADC (12-bit) instance.
adc = Adafruit_ADS1x15.ADS1015()

# Get I2C bus
bus = smbus.SMBus(1)

#GPIO Mode (BOARD / BCM)
```

```

GPIO.setmode(GPIO.BOARD)
GPIO.setwarnings(False)

#set GPIO Pins
GPIO_TRIGGER = 32
GPIO_ECHO = 36
RED = 33
BLUE = 35
GREEN = 37

#set GPIO direction (IN / OUT)
#Ultrasonic Sensor
GPIO.setup(GPIO_TRIGGER, GPIO.OUT)
GPIO.setup(GPIO_ECHO, GPIO.IN)

#RGB LED
GPIO.setup(RED, GPIO.OUT)
GPIO.output(RED, 0)
GPIO.setup(GREEN, GPIO.OUT)
GPIO.output(GREEN, 0)
GPIO.setup(BLUE, GPIO.OUT)
GPIO.output(BLUE, 0)

p_R = GPIO.PWM(RED, 2000) # set Frequece to 2KHz
p_G = GPIO.PWM(GREEN, 2000)
p_B = GPIO.PWM(BLUE, 5000)

p_R.start(0) # Initial duty Cycle = 0(leds off)
p_G.start(0)
p_B.start(0)

##### ULTRASONIC DISTANCE #####
def distance():
    # set Trigger to HIGH
    GPIO.output(GPIO_TRIGGER, True)

    # set Trigger after 0.01ms to LOW
    time.sleep(0.00001)
    GPIO.output(GPIO_TRIGGER, False)

    StartTime = time.time()
    StopTime = time.time()

    # save StartTime
    while GPIO.input(GPIO_ECHO) == 0:
        StartTime = time.time()

    if (StartTime-StopTime) >= 3:
        distance = 0.5
        return distance

    # save time of arrival
    while GPIO.input(GPIO_ECHO) == 1:
        StopTime = time.time()

```

```

# time difference between start and arrival
TimeElapsed = StopTime - StartTime
# multiply with the sonic speed (34300 cm/s)
# and divide by 2, because there and back
distance = (TimeElapsed * 34300) / 2

return distance

##### TEMPERATURE #####
def temperature():
    tempStore = open("/sys/bus/w1/devices/28-0000095f774a/w1_slave")
    data = tempStore.read()
    tempStore.close()
    tempData = data.split("\n")[1].split(" ")[9]
    temp = float(tempData[2:])
    temp = temp/1000.0
    return temp

##### RGB LED #####
def map(x, in_min, in_max, out_min, out_max):
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min

def setColor(r,b,g):
    R_val = r
    G_val = g
    B_val = b

    R_val = map(R_val, 0, 255, 0, 100)
    G_val = map(G_val, 0, 255, 0, 100)
    B_val = map(B_val, 0, 255, 0, 100)

    p_R.ChangeDutyCycle(R_val) # Change duty cycle
    p_G.ChangeDutyCycle(G_val)
    p_B.ChangeDutyCycle(B_val)

    return (R_val, G_val, B_val)

##### LUMINOSITY #####
def luminosity():
    # TSL2561 address, 0x39(57)
    # Select control register, 0x00(00) with command register,
    0x80(128)
    # 0x03(03) Power ON mode
    bus.write_byte_data(0x39, 0x00 | 0x80, 0x03)
    # TSL2561 address, 0x39(57)
    # Select timing register, 0x01(01) with command register,
    0x80(128)
    # 0x02(02) Nominal integration time = 402ms
    bus.write_byte_data(0x39, 0x01 | 0x80, 0x02)

```

```

time.sleep(0.5)

# Read data back from 0x0C(12) with command register, 0x80(128),
2 bytes
#          ch0          LSB,          ch0          MSB
data  = bus.read_i2c_block_data(0x39, 0x0C | 0x80, 2)

# Read data back from 0x0E(14) with command register,
0x80(128), 2 bytes
#          ch1          LSB,          ch1          MSB
data1 = bus.read_i2c_block_data(0x39, 0x0E | 0x80, 2)

#          Convert          the          data
ch0  = data[1] * 256 + data[0]
ch1  = data1[1] * 256 + data1[0]
ch2  = ch0-ch1

return (ch0,ch1,ch2)

#####          FORCE          SENSOR          #####
def force():

    #Gain          Value
    GAIN = 1

    #Input          Channel          Variable
    AI    = 1
    Vin   = 4.096
    R     = 10000.0

    #          Find          ADC          value
    ADC = adc.read_adc(AI, gain=GAIN)

    #          Convert          to          voltage
    Vfsr = (ADC/2047.0)*Vin # (ADC/bits) = (Voltage/input
voltage)

    #Calculate          Force
    #force = 0.0000689*math.exp(4.27*Vfsr)
    if Vfsr <= 0.875510204 :
        force = 0.023294217*Vfsr
    elif Vfsr <= 2.087755102 :
        force = 0.067294403*Vfsr
    elif Vfsr <= 3.030612245 :
        force = 0.973365476*Vfsr
    else:
        force = 34.06779166*Vfsr

    return (ADC,Vfsr,force)

#####          MAIN          #####
#          Declare          array          variables
dist, temp = [0]*10, [0]*10
fullspec, ir, vis = [0]*6, [0]*6, [0]*6

```

```

a,          v,          f          =          [0]*10,[0]*10,[0]*10

for          x          in          range(0,10):
    #          Collect          Data
    dist[x] = distance()
    print    ("Measured    Distance:    %.1f    cm"    %    dist[x])

    temp[x]          =          temperature()
    print    ("Temperature:    %.2f    degrees    Celsius"    %    temp[x])

    if          x          <          6:
        setColor(R[x],G[x],B[x])
        fullspec[x],ir[x],vis[x]          =          luminosity()
        vis[x]          =          vis[x]/30.0*100
        print    "Full Spectrum(IR + Visible): %d lux" %fullspec[x]
        print    "Infrared    Value:    %d    lux"    %ir[x]
        print    "Visible    Value:    %d    lux"    %vis[x]

    a[x],v[x],f[x]          =          force()
    print    ('ADC          value:    %.2f'    %a[x])
    print    ('FSR          voltage:    %.2f    V'    %v[x])
    print    ('Force:    %.2f    kg'    %f[x])
    print('=====')

    time.sleep(1)

    #          Clean          up          GPIO          board
    p_R.stop()
    p_G.stop()
    p_B.stop()
    GPIO.cleanup()

    #          Calculate          Average
    sumTemp          =          0
    sumDist          =          0
    sumForce          =          0
    for          y          in          range(10):
        sumTemp          +=          temp[y]
        sumDist          +=          dist[y]
        sumForce          +=          f[y]
    avgTemp          =          sumTemp/10.0
    avgDist          =          sumDist/10.0
    avgForce          =          sumForce/10.0

    #Write          to          file
    f=open("sensor.txt",          "w+")
    f.write("%.2f    %.2f    %.2f"    %(avgTemp,avgDist,avgForce))
    f.write("\n6\n")
    for          n          in          range(6):
        f.write("%d    %d\n"    %(wavelength[n],vis[n]))
    f.close()

```

January 5th, 2019: Output

Because of the planks extending the arms of the motor, the screw still could not hold the motor together with the rotating base. Additionally, the motor system was mildly damaged due to all the drilling into it, which caused the holes to be slanted and generally just not good. As a result, the bottom motor was changed to a new one, and the new screws that came with the motor were used. The group attempted to get rid of the long planks underneath the rotating base, as the pillars underneath the base seemed quite successful by themselves at keeping the base stable. As well, this would mean the screw would finally be long enough to keep the motor intact. This was fixed, and the motor system could be kept together without the planks of wood. This was tested by lifting the base up by just the rotating part of the base.

However, with the long planks of wood not included anymore, the height from the rotating part of the base to the stationary base became shorter, as the planks added height. This means that the height of the pillars also had to be shortened. This was done by sawing through the wood. The pieces seemed to be significantly shorter than needed, and thin pieces of wood were added to the bottom to increase the height and make the rotation of the base smooth. Folded pieces of cardstock were used when there was a very small height that needed to be filled. Upon doing this, the arm was very stable and ready to be used.

Software Output Screen

To program a graphic user interface to display the data collected by the sensors, Python's Kivy library was used. Although Kivy is an open source Python library used for developing mobile apps for touch screen monitors, the library can also run applications on the computer.

A static page was created to display the output data. After analyzing the capability and features of the library, the team agreed on using the BoxLayout to create the graphic user interface. The other choice had been flowlayout. Since the boxlayout provided a way to program the GUI with respect to sequence and orientation (vertical ordering vs horizontal ordering) while flowlayout was unrestricted (more potential to flaws), the team decided that the boxlayout would provide a easier and more adaptable user interface.

While planning out the user interface for the output data, the team brainstormed all the necessary data values to display.

The page had the following features:

Labels	Function
Light Spectra	In order to create the graph for the light spectra, matplotlib.h is used to create the plot, and save it as an image. The newly saved image is then called upon to display as an image label.
Soil Image	An image of the soil sample will be taken and a colour reference scale will be placed under the image for reference.
Temperature	The temperature data is collected from the text file and is then displayed onto the user interface, which is rounded to at most three significant digits.

Volume	The volume data is collected from the text file and is then displayed onto the user interface, which is rounded to at most three significant digits.
Mass	The mass data is collected from the text file and is then displayed onto the user interface, which is rounded to at most three significant digits.
Density	Density is calculated by dividing mass by volume, which was then rounded to the first significant digit.
Menu	The menu button will return to the beginning of the program -- the camera livestream. It has yet to be implemented
Close	The close button will close the application.

Fig 22. Displays on the Output Screen. The calculated and measured mass, volume, density, temperature, reflective spectra, as well as the imaged soil, menu and close button will be displayed on the graphical user interface.

The application will look like the following:

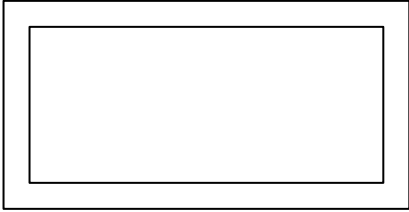
Values	
Light	
Temperature	
Mass	
Volume	
Density	
Menu	Close

Fig 23. Format of the Output Screen. The output screen displays the light spectra, temperature, mass, volume, and density. These values are retrieved from a text file that is created by the previous program that tests the soil sample.

To run the program through command prompt, the command should be:

```
>> py OutputValues.py demofile.txt
```

Similar to a java script, OutputValues4.py is the program to be runned, and demofile.txt is the string parameter of the path name. The text file title “demofile.txt” should be the file

written by the sensor program, where the text file should be formatted in the following manner:

```
0 *****Text file format*****
1 [temp] [volume] [mass]
2 [number_of_coordinates]
3 [x] [y]
4 [x] [y]
5 [x] [y]
6 [etc]
```

A snippet of the text file might look like this:

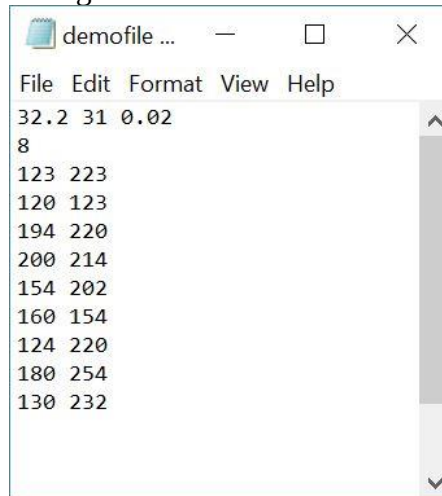


Fig 23. Text File Output. The values that are measured by the sensors are in the following text file, and will be inputted into the graphical user interface.

The code for the output data interface is the following:

```
-----
# imports
from kivy.app import App
from kivy.core.window import Window
from kivy.uix.boxlayout import BoxLayout
from kivy.uix.button import Button
from kivy.uix.label import Label
from kivy.logger import Logger
from kivy.uix.image import Image

import sys
import numpy as np
import matplotlib.pyplot as plt

    #take data from text file
def values(path):
    global tem
    global vol
```

```

global mas
global den
f = open(path, "r")
vals = f.readline().split(" ")
tem = round(float(vals[0]),3)
vol = round(float(vals[1]),3)
mas = round(float(vals[2]),3)
den = round(mas/vol,3)

#adding coordinates
x = []
y = []
for i in range(int(f.readline())):
    s = f.readline()
    coord = s.split(" ")
    x.append(int(coord[0]))
    y.append(int(coord[1]))

# Plot
colors = (0,0,0)
area = np.pi*3
plt.scatter(x, y, s=area, c=colors, alpha=0.5)
plt.xlabel('x')
plt.ylabel('y')
plt.savefig('lightPlt')

# Boxlayout is the App class
class outputPage(BoxLayout):
    global img
    def __init__(self,**kwargs):
        super(outputPage,self).__init__(**kwargs)

        self.orientation='vertical' #possible for error

        photoBox = BoxLayout(orientation='horizontal')
        photoTitle = Label(text = "Light Spectra:")
        img = Image(source = "lightPlt.png")
        photoPlot = img
        photoBox.add_widget(photoTitle)
        photoBox.add_widget(photoPlot)

        verticalBox = BoxLayout(orientation='vertical')

        #temperature
        temperature = BoxLayout(orientation='horizontal')
        tempTitle = Label(text = "Temperature:")
        tempValue = Label(text = ""+str(tem))

```

```

temperature.add_widget(tempTitle)
temperature.add_widget(tempValue)

#volume
volume = BoxLayout(orientation='horizontal')
volTitle = Label(text = "Volume:")
volValue = Label(text = ""+str(vol))
volume.add_widget(volTitle)
volume.add_widget(volValue)

#mass
mass = BoxLayout(orientation='horizontal')
massTitle = Label(text = "Mass:")
massValue = Label(text = str(mas))
mass.add_widget(massTitle)
mass.add_widget(massValue)

#density
density = BoxLayout(orientation='horizontal')
densTitle = Label(text = "Density:")
densValue = Label(text = str(den))
density.add_widget(densTitle)
density.add_widget(densValue)

verticalBox.add_widget(temperature)
verticalBox.add_widget(volume)
verticalBox.add_widget(mass)

horizontalBox = BoxLayout(orientation='vertical')
button1 = Button(text="Menu")
button1.bind(on_press=self.agafunc)
button3 = Button(text="Close")
button3.bind(on_press=self.clsfunc)
horizontalBox.add_widget(density)
horizontalBox.add_widget(button1)
horizontalBox.add_widget(button3)

self.add_widget(photoBox)
self.add_widget(verticalBox)
self.add_widget(horizontalBox)
def clsfunc (self,obj):
    App.get_running_app().stop()

def agafunc (self,obj):
    Window.close()

class Application(App):

```

```

def on_stop(self):
    Logger.critical('Goodbye')
def build(self):
    return outputPage()

# Instantiate and run the kivy app
if __name__ == '__main__':
    values(sys.argv[1])
    Application().run()

```

The output will look like:

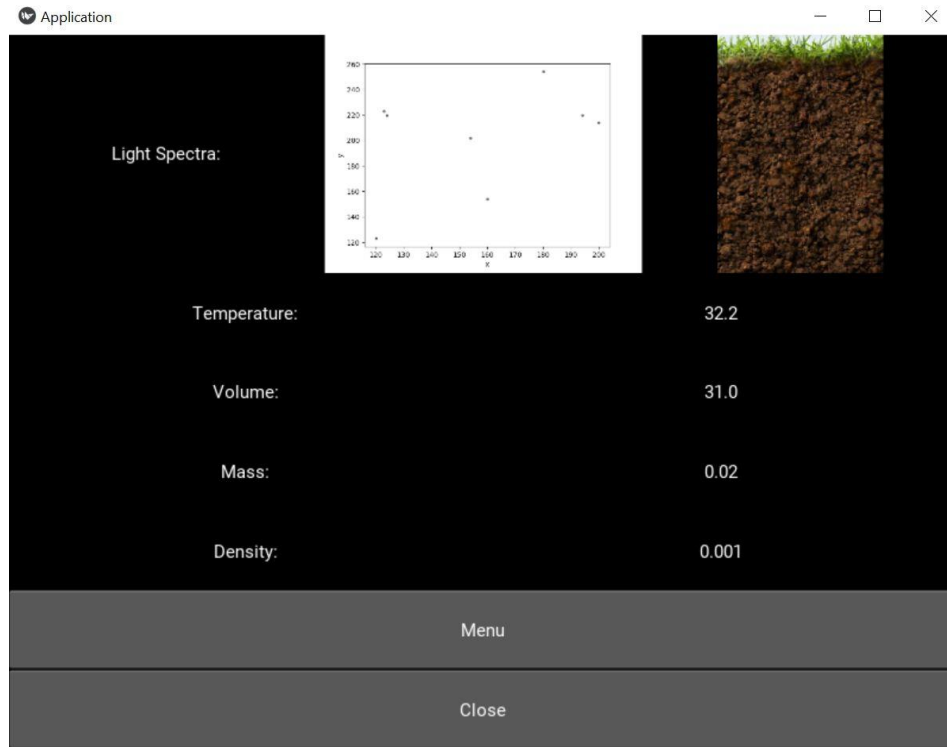


Fig 24. Screenshot of the Display Window. The output program was created using Python's Kivy library which is similar to Java's GUI library. The window was created with a boxlayout which provides a sleeker design.

Currently, the Kivy application successfully reads the text file and is able to display the test data. However, there were a few technical issues that the group came across. Firstly, the display window is not resizable. This is not a major issue as the current window is an appropriate size that does not require resizing. Secondly, the menu button still doesn't work; it closes the application rather than restarting the application. However, since the soil sampler only needs to run once, the group decided that the menu button is unnecessary and agreed to remove it in the next version. Finally, the label titled "Light Spectra" is not aligned with the rest of the labels below it. While this is not a major issue, the group decided to readjust the dimensions of the labels for the next version.

January 6th, 2019: Output

The arm was damaged to a moderate degree. The entire structure fell sideways, and the base rotating plate became separated from everything else. This meant that most of the items on the base had to be unscrewed and rescrewed, so that the screw connecting the base motor to the rotating plate could be reconnected. This was done. As well, a hook/ring was added in front of the spoolie of string, so that the string would feed into the spoolie cleanly and not rub against the top of the spoolie. This would cause friction and may break it. The top of the spoolie was further sanded and cut down in size, because sometimes the string would still rub against the top of the spoolie, making the unravelling or re-ravelling more difficult.

Software: Output Display

In the new version, the window's title was changed to "Zoroak Values" to provide a more specificity. As well, the layout was rearranged to align the label "Light Spectra:" with the rest of the measurement labels. Units were also added onto the Temperature, Volume, Mass, and Density labels to provide clarity to the user. Due to the fact that the soil sample will have small dimensions, smaller units (g, cm³, g/cm³) were used in place of the standard unit of notations (kg, m³, kg/m³). Finally, the menu button was removed as the program does not need to run again after the first testing sample.

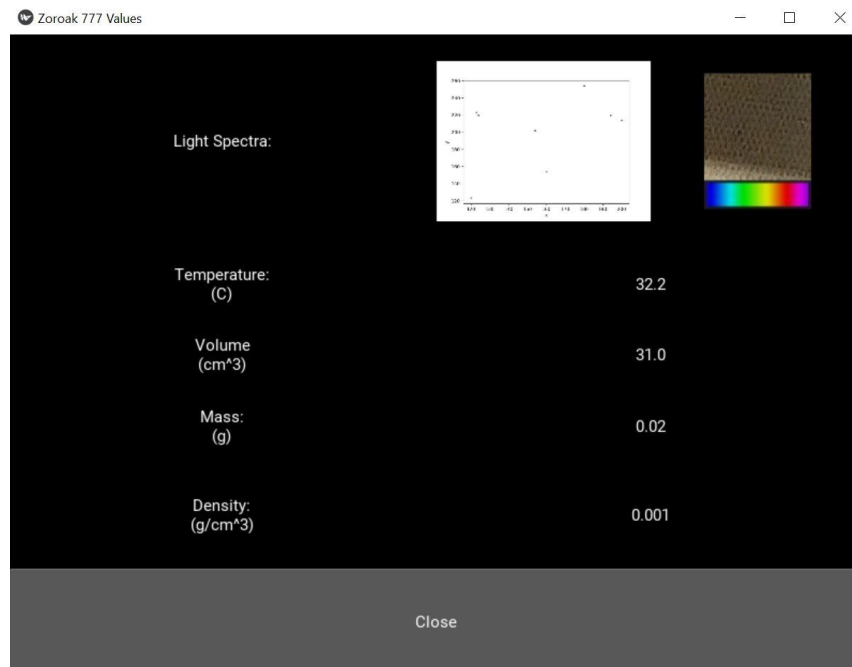


Fig 25. Output Screen. This features a clean black background, a large close button, the measured and calculated properties of the soil, and the two images for the reflective spectra and the soil sample.

January 7th, 2019: Testing Station

Unfortunately, the arm was slightly damaged again. The base broke off once again, indicating that an improperly-sized screw was used to connect the base motor to the rotating plate. Everything on the rotating base was unscrewed again, and a more suitable screw was found and connected to the base servo. Once this was done, all the other components of the arm were reattached to the base. However, after a few minutes, the arm was mildly damaged again. This time, the casualties were less severe. One of the screws at the bottom of the base became loose, causing the entire arm to wobble slightly. This was fixed simply by screwing in the bottoms screws more.

The group designed a testing box, which will be the designated area used to measure the characteristics of the soil. Ideally, once the arm collected the soil from the sample site, the claw will deposit its sample to the testing box, where the sample soil will pass through the funnel into the interior.

Drawings were made of the testing box.

The front view of the test box:

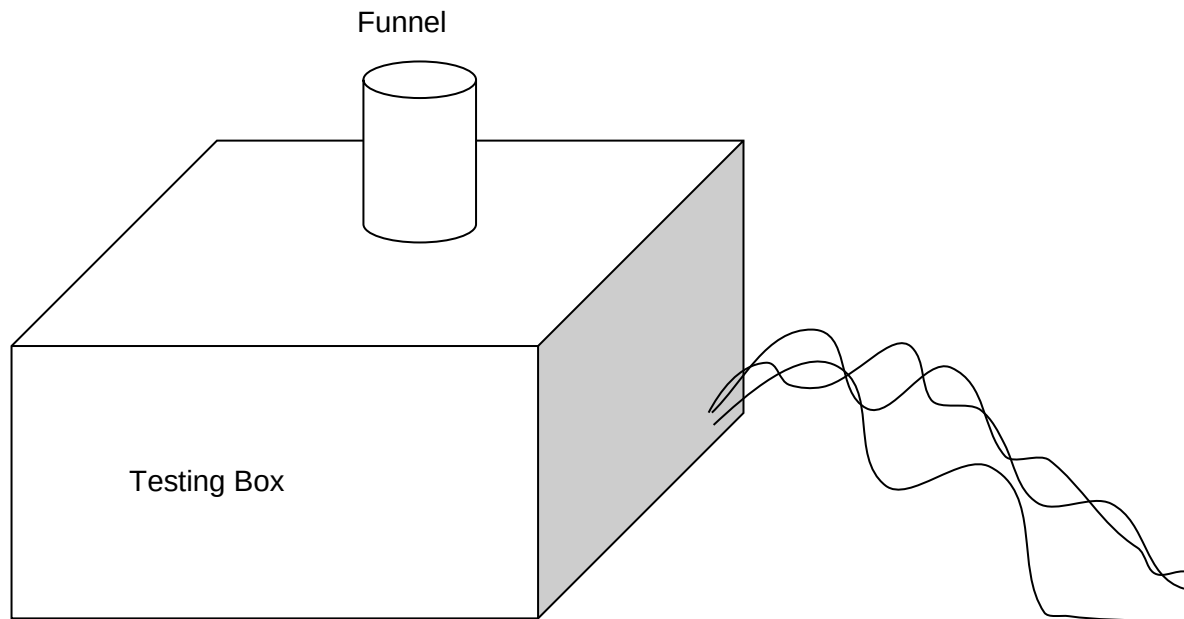


Fig 26. Testing Box. The sensors are installed in the interior of the testing box. The soil collected by the arm will be deposited into the box through the funnel. The box is wired to the raspberry pi and used to test the soil sample.

With the sensors and equipment on the interior of the testing box, it provides a sleeker design with less visible clutter, providing the user with a much better experience.

A top view of the testing box:

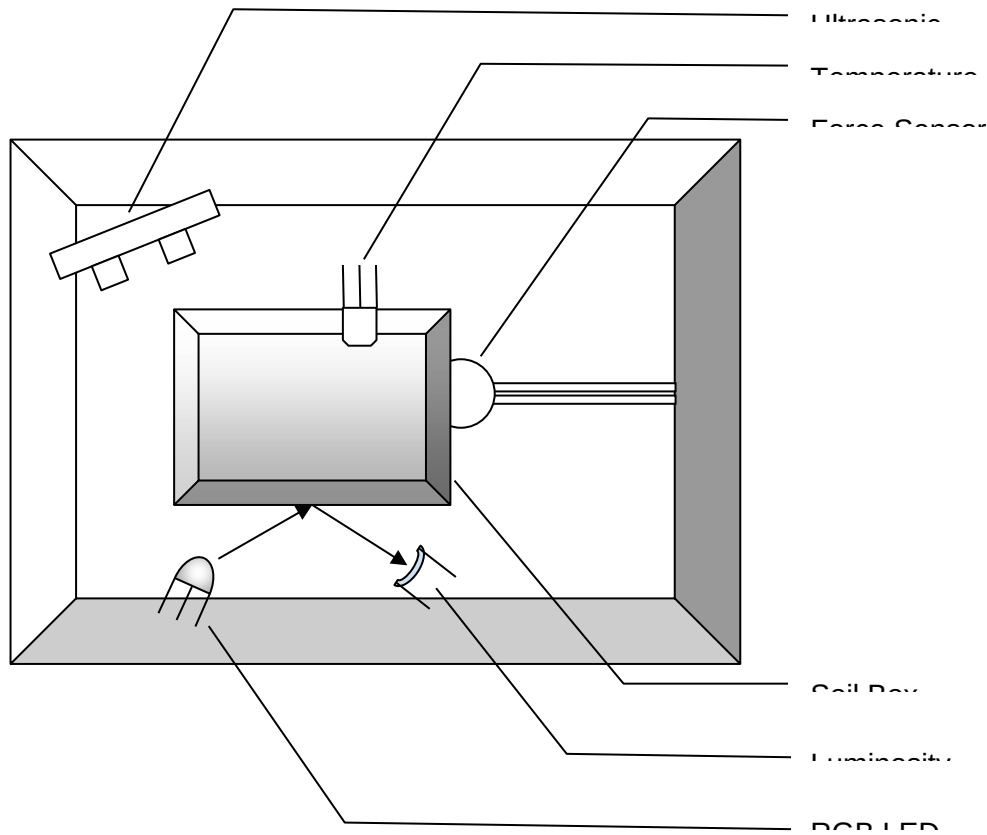


Fig 27. Testing Box (Bird's eye view). The interior of the testing box will have sensors installed inside: the ultrasonic sensor, temperature sensor, force sensor, luminosity sensor, and rgb led. A smaller box is placed inside of the testing box to collect the soil. The ultrasonic sensor is used to measure the depth of the soil. The temperature sensor is used to measure the temperature of the soil. The force sensor is used to measure the weight of the soil. The luminosity sensor and RGB LED were used to measure the light spectra.

The sensors and equipment will be installed into the interior of the testing box. The data testing occurs inside:

Sensor	Function
Light Spectra	The light spectra graph will be created from a set of measured points. As the light spectra graph requires coordinate points where $(x,y) \rightarrow (\text{wavelength, light intensity})$, the luminosity sensor and rgb light are used to obtain coordinates.
Luminosity sensor	- Measures the luminosity of the light that is reflected off the soil from the RGB LED
RGB LED	- Emits different color lights of different wavelengths (550-700 nm). The rays are shone onto the soil and reflected towards the luminosity sensor.

Temperature	The temperature is calculated in celsius degrees
Temperature Sensor	- Temperature sensor is installed near the soil box to measure the air of the temperature above the soil
Mass	The mass of the soil sample is calculated by measuring the weight of the soil sample and the knowledge of Newton's Second Law ($F=ma$)
Force Sensitive Resistor	- The force sensor is placed underneath the soil box to measure the weight of the box
Volume	The volume of the soil sample was measured using mathematical formulas. With the precalculated base area of the soil box, the height must be found in order to calculate the volume of the soil
Ultrasonic sensor	- The ultrasonic sensor is installed at the roof of the soil box, perpendicularly facing the surface of the soil. The measurement will be used to calculate the height of the soil sample, which is then used to calculate the volume.

Fig 28. The properties of soil and the sensors used to test each property. Using the laws of physics, each property of soil is defined, and each sensor's abilities are matched accordingly.

A prototype was made using cardboard. Duct tape was used to temporarily hold things in place. The box was covered with a lid to ensure that there would be no external light to interfere with the measurements made for the light spectra. As well, there was a hole on the top of the box in order to place the funnel. A funnel was added to the box to provide a easy method of pouring soil into the soil box. Because there exists a hole and therefore an external light source, the light spectra will be affected. However, since the external light source is not very large in comparison to the brightness of the LED light, the effect will not be very significant.



Fig 29. Soil Testing Station Prototype. A cardboard box was used as the prototype soil testing station. Holes were cut to allow wires to pass through. All the sensors and their circuit boards were inside the box, but not attached to the box itself.

After determining the dimensions of the soil testing box and the locations of each sensor, the group considered on creating a sturdier and more aesthetically pleasing box.

A wooden box was crafted for the testing box. Its dimensions are 12x14x20 cm. The sensors and equipment will later be installed into the box.



Fig 30. A wooden box. The wooden box will be used for the testing box. This provides a sturdier and more aesthetically pleasing design.

Power supply issues

Due to the servos being tested individually for ensure full functionality, it was a fair assumption that they would work concurrently in a parallel circuit using an external voltage. However, as the battery drained, signaled by the eventually dimming of a light emitting diode, more voltage was required.

Due to the modification and implementation of a high torque servo for base rotation, this required further current draw from the power source. Upon testing and troubleshooting via a separate, unique individual power source, it was first presumed that a cold solder joint or wire failure was the culprit at hand. It was eventually discovered that it was a direct result of a lack of current after testing several combinations of servos pulsing whilst disconnecting an alternative permutation.

January 8th, 2019: Servo Code + Testing Servos on Arm

When developing the servo arm, the group decided to program the software and servo motion in polar coordinates for easier measurements. The group also realized that the weight of the arm became significantly greater for the servos closer the the base. The servos holding more weight require more power and torque, which explains why the servos need to be stronger as they are closer to the base.

The arm was created to have four degrees of freedom. The set of servos comprised of:

Servos	Type	Function
Base Servo	180 High Torque	The base servo would pivot the base of the arm to orient the entire arm towards the soil sample's direction. The base servo controls the theta of the coordinates
Spool Servo	Continuous	The spool unwinds and rewinds to allow for the main segment of the arm to go down or come up. This carries the weight of the arm so it requires an ample amount of force. This provides movement in the x-y direction.
Forearm Servo	180 Standard	The forearm servo will allow the arm to pivot 180 around the end of the main arm. This provides movement in the x-y direction.
Claw Servo	180 Micro	The claw servo is small and requires less power. This is due to the fact that it only needs to pick up soil and is on the end of the end, thus requiring the least amount of energy.

Fig 31. List of Servos. The arm was comprised of four servos. Each servo had different characteristics based on its purpose and function.

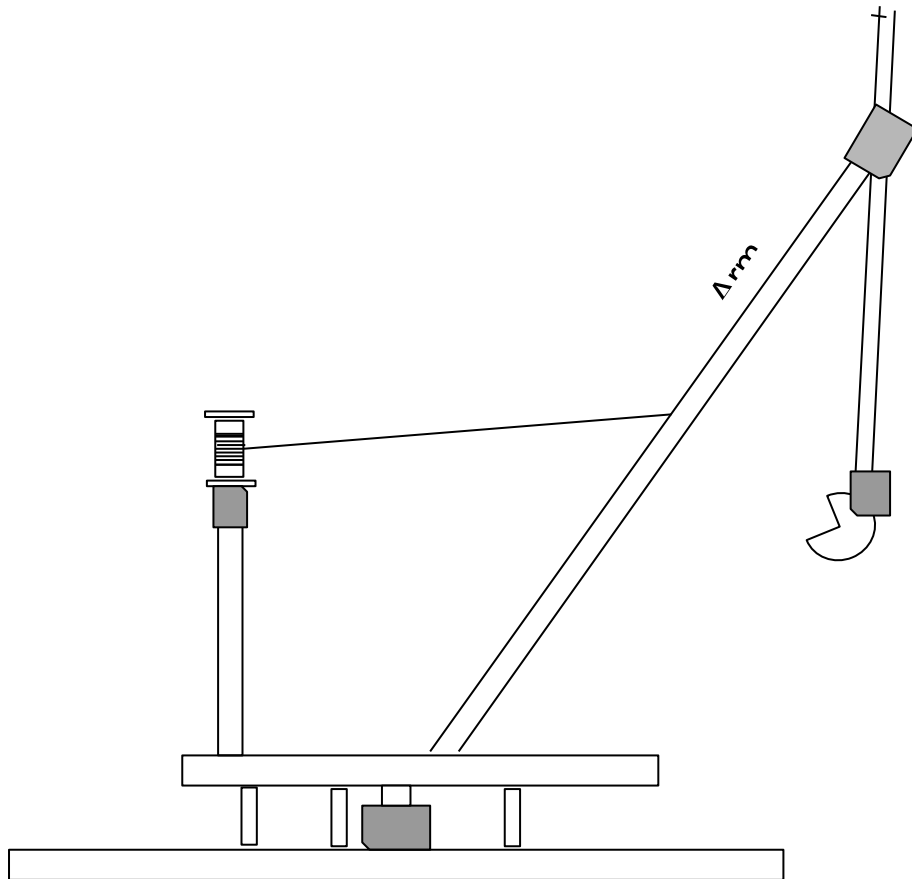


Fig 32. Diagram of the entire arm. The arm features a 3-joint system, similar to a human arm. There are servos at each joint to move the arm up and down. There is a servo motor at the bottom turning a lazy susan, which moves the arm left and right.



Fig 33. Picture of Arm.

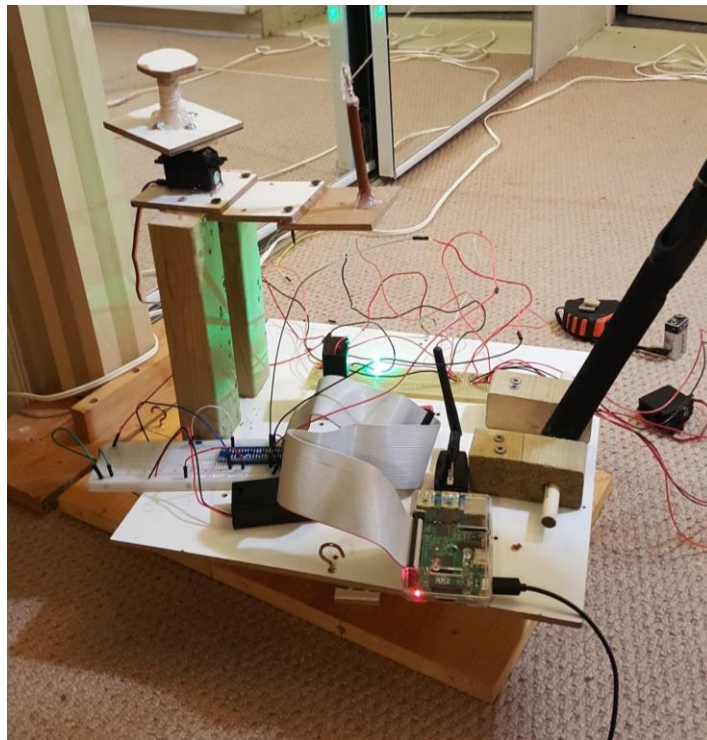


Fig 34. Base of arm

The group noticed the clutter at the bottom of the base. It was decided that the base will be rearranged once the software for the arm is debugged and ready. The GPIO pins will also be soldered into a PCB board. However, this was considered last priority as the group had to finish debugging the code for servo motion to occur.

The final servo code implemented the data from the live stream to determine the position of the the arm relative to the target at which the user clicked for where the soil would be sampled. The base servo motor then moves to the proper angle in line with the soil sample. The code then calculates the angle of the forearm servo using trigonometry and cosine law and then lowers the long arm using the spool servo until the proper depth given the depth detection data from the livestream. After scooping the soil with a clean movement, the code then rotates to the position of the testing station using the coordinates given previously by the user and the same principles. The final servo code is shown below:

```

from sys import stdin, stdout
from math import *
import RPi.GPIO as GPIO
import time
import requests

# angl, ang2, ang3, ang4 -> angle of long rod, angle of small rod, angle of
base, angle of string
# EVERYTHING IS IN METRES

webaddress = 'http://192.168.2.48:8000/coordinates'
x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur = 0.0, 0.0, 0.0,
0.0, 90.0, 0.0, 0.0
len1, len2 = 0.985, 0.40
heightBase, heightFunnel = 0.135, 0.43
depthFin = 1.3
angArmCur, angArmFin = 0.0, 0.0
angBigArmFin = 0.0
angScoop = 0.0
heightOrg = 1.015
global angBase
angBase = 0.0
testAng, testDist, angArmSoil, angBigArmSoil = 0.0, 0.0, 0.0, 0.0

print("Start Arm Movement")

def get():
    r = requests.get(webaddress)
    data = r.json()

    x = float(data['coordinates']['x'])
    y = float(data['coordinates']['y'])
    angBaseFin = float(data['coordinates']['xyang'])
    distFin = float(data['coordinates']['xydist'])
    angBaseCur = float(data['coordinates']['Bang'])
    distCur = float(data['coordinates']['Bdist'])
    depthCur = float(data['coordinates']['depth'])
    testAng = float(data['coordinates']['Tang'])
    testDist = float(data['coordinates']['Tdist'])

    print(x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, testAng,
testDist)
    distFin -= 0.15
    distSoil = testDist

```

```

distSoil      ==      0.15
temp          =      distSoil
distSoil = sqrt(distSoil ** 2 + (heightBase - heightFunnel) ** 2)
distFin    = sqrt(distFin ** 2 + heightBase ** 2)
angArmSoil = acos((pow(len1, 2) + pow(len2, 2) - pow(distSoil, 2)) / (2
*      len1      *      len2))
angBigArmSoil = acos((pow(len1, 2) + pow(distSoil,2) - pow(len2, 2)) /
(2      *      len1      *      distSoil))
angArmFin = acos((pow(len1, 2) + pow(len2, 2) - pow(distFin, 2)) / (2
*      len1      *      len2))
angBigArmFin = acos((pow(len1, 2) + pow(distFin,2) - pow(len2, 2)) / (2
*      len1      *      distFin))
angBigArmSoil +=      acos(temp      /      distSoil)
angArmFin      *=      180.0      /      pi
angArmFin      +=      3
angArmSoil      *=      180.0      /      pi
angArmSoil      -=      6

angBaseFin      *=      (-0.44*sin(angBaseFin/180*pi*2)+1)
angBaseFin      -=      4
testAng      *=      (-0.20*sin(testAng/180*pi*2)+1)
testAng      -=      4

#angArmFin      -=      10.0

#print("b",      angArmFin)
#return(angArmFin)

return x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur,
angArmFin, angBigArmFin, testAng, angArmSoil, angBigArmSoil

def      Move360(angCur,      angFin):
    if      angCur      <      angFin:
        continuous.ChangeDutyCycle(7.53)
        time.sleep(0.05)
        print("a")
    else:
        continuous.ChangeDutyCycle(6.48)
        time.sleep(0.05)
        print("b")

def      Move180(servo,      angCur,      angFin,      interval,      tm):
    if      servo      ==      base:
        global      angBase
    if      angCur      >      angFin:
        angBase      -=      interval
        servo.ChangeDutyCycle(angBase      /      18.0      +      2.5)
        time.sleep(tm)
    else:
        angBase      +=      interval
        servo.ChangeDutyCycle(angBase      /      18.0      +      2.5)
        time.sleep(tm)
    return      angBase

```

```

if angCur > angFin:
    angCur -= interval
    servo.ChangeDutyCycle(angCur / 18.0)
    time.sleep(tm)
else:
    angCur += interval
    servo.ChangeDutyCycle(angCur / 18.0)
    time.sleep(tm)
return angCur

GPIO.setmode(GPIO.BOARD)

# Set-up Servos
pin_continuousservo = 11 # GPIO 17 #6.93 keeps it steady
pin_big servo      = 12 # GPIO 18
pin_claw servo     = 13 # GPIO 27
pin_baseservo      = 38 # GPIO 20

pin_bottomSwitch = 15
pin_topSwitch    = 23 # GPIO 11

try:
    GPIO.setup(pin_continuousservo, GPIO.OUT) # continuous servo
    GPIO.setup(pin_big servo, GPIO.OUT) # big servo
    GPIO.setup(pin_claw servo, GPIO.OUT) # claw servo
    GPIO.setup(pin_baseservo, GPIO.OUT) # base servo

    GPIO.setup(pin_bottomSwitch, GPIO.IN, pull_up_down=GPIO.PUD_UP)
    GPIO.setup(pin_topSwitch, GPIO.IN, pull_up_down=GPIO.PUD_UP)

    continuous = GPIO.PWM(pin_continuousservo, 50)
    big = GPIO.PWM(pin_big servo, 50)
    claw = GPIO.PWM(pin_claw servo, 50)
    base = GPIO.PWM(pin_baseservo, 50)

    # set servos to default
    big.start(0.0) # ninety degrees #23 degree difference
    claw.start(0.0) # ninety degrees
    base.start(0.0) # ninety degrees
    continuous.start(0)

    angBase = 110

    #x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _, _ = get()
    #if angBaseCur > angBaseFin:
    #    angBase = 120.0

    #base.ChangeDutyCycle(12.5)
    #time.sleep(5)
    #print("life is life")
    #for i in range(300):
    #    #print(angBase)
    #    #angBase = Move180(base, angBaseCur, angBaseFin, 0.2, 0.05)
    #    #print("wut")

```



```

#time.sleep(100)
#continuous.ChangeDutyCycle(6.43)
#time.sleep(2)

state = False
while True:
    x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _, _, _ = get()
    angBase = Move180(base, angBaseCur, angBaseFin, 0.2, 0.01)
    print(angBase, angBaseFin, angBaseCur)
    time.sleep(0.05)
    if (state == True):
        if (abs(angBaseCur-angBaseFin) <= 0.75):
            break
        else:
            state = False
    elif (abs(angBaseCur-angBaseFin) <= 0.75):
        state = True

#base.ChangeDutyCycle(11.8)
#time.sleep(5)

base.ChangeDutyCycle(0.0)
time.sleep(2)

#print("hooray!")

#base.stop()

x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, angArmFin, _, _ = get()
print("dist", distFin)
print("angArmFin", angArmFin)

angArmFin = ((angArmFin+6)-24.154)/1.1769

while (abs(angArmCur-angArmFin) >= 0.2):
    angArmCur = Move180(big, angArmCur, angArmFin, 0.2, 0.01)

#print("hooray!x2")

time.sleep(2)

x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _, angBigArmFin, _, _ = get()

heightFin = len1*sin(angBigArmFin)+0.15
print(heightFin)

while (depthCur > heightFin):
    x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _, angBigArmFin, _, _ = get()

```

```

        print(depthCur, heightFin)
        Move360(depthCur, heightFin)

    Move360(2.0, 1.0)
    time.sleep(2.0)

    continuous.ChangeDutyCycle(0.0)
    time.sleep(2)

    claw.ChangeDutyCycle(2.5)
    time.sleep(2)

    claw.ChangeDutyCycle(0.0)

    while (depthCur < 0.95):
        x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _,
angBigArmFin, _, _ = get()
        print(depthCur, 0.95)
        Move360(depthCur, 0.95)

    continuous.ChangeDutyCycle(6.93)
    time.sleep(2)

    print("tada")

    state = False
    while True:
        x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _, _
testAng, _ = get()
        angBase = Move180(base, angBaseCur, testAng, 0.2, 0.01)
        print(angBase, testAng, angBaseCur)
        time.sleep(0.05)
        if (state == True):
            if (abs(angBaseCur-testAng) <= 0.75):
                break
            else:
                state = False
        elif (abs(angBaseCur-testAng) <= 0.75):
            state = True

    base.ChangeDutyCycle(0)
    time.sleep(2)

    x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, angArmFin, _
testAng, angArmSoil, angBigArmSoil = get()
    print("dist", distFin)
    print("angArmSoil", angArmSoil)

    angArmSoil = (angArmSoil-24.154)/1.1769

    while (abs(angArmCur-angArmSoil) >= 0.2):
        angArmCur = Move180(big, angArmCur, angArmSoil, 0.2, 0.01)

    print("hooray!x2")

```

```

big.ChangeDutyCycle(0.0)
time.sleep(2)

x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _,
angBigArmFin, testAng, angArmSoil, angBigArmSoil = get()
print("a", angBigArmSoil)
heightSoil = len1*sin(angBigArmSoil) + 0.2
print(depthCur, heightSoil)

while (depthCur > heightSoil):
    x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _,
    angBigArmFin, testAng, angArmSoil, angBigArmSoil = get()
    print("qq", depthCur, heightSoil)
    Move360(depthCur, heightSoil)

print("bob")

Move360(2.0, 1.0)
time.sleep(0.3)

continuous.ChangeDutyCycle(6.93)
time.sleep(2)

#claw = GPIO.PWM(pin_clawservo, 50)
claw.start(0.0)
claw.ChangeDutyCycle(12.5)
time.sleep(2)

claw.ChangeDutyCycle(0.0)

print("hmmm")

while (depthCur < 0.95):
    x, y, angBaseFin, distFin, angBaseCur, distCur, depthCur, _,
    angBigArmFin, _, _ = get()
    print(depthCur, 0.95)
    Move360(depthCur, 0.95)

    continuous.ChangeDutyCycle(0.0)
#continuous.ChangeDutyCycle(7.5)
#time.sleep(2)

GPIO.cleanup()

except KeyboardInterrupt:
    base.stop()
    claw.stop()
    continuous.stop()
    big.stop()
    print("stop")
    GPIO.cleanup()

```

January 9th, 2019: Integration of All Code

The code was all integrated into one bash script that will be run by the user. This script will first run the user interface for the user to specify the location of the soil sample and then remotely connect to the Raspberry Pi to move the arm and collect sensor data from the soil. The data will then be transferred back to the computer and displayed on a GUI. The following shows the final bash script:

```
#!/bin/bash

python3                                Webcam2.py                                &
python3                                webserver.py                                &

#echo                                "Running                                bash                                script"

while                                true;                                do
    if                                [                                -e                                start.txt                                ];                                then
        echo                                "START"
        break
    fi
done

rm                                start.txt
echo                                "START"
echo                                "Connecting                                to                                Raspberry                                Pi"
ssh pi@192.168.2.63 'cd APP; echo "connected to Raspberry Pi"; python3
test2.py;                                python                                ALLSENSORS.py;                                exit'

sftp                                pi@192.168.2.63:/home/pi/APP/sensor.txt

python3 OutputValues6.py sensor.txt
```

January 10th, 2019: Installing Soil Testing Station

The sensors and circuit boards were installed into the soil analyzing station. Firstly, holes were drilled in appropriate locations on the top of the box, to allow the funnel and wires to pass in/out of the box. As well, because the ultrasonic sensor needs to be placed orthogonally to the box lid, a thin groove only thick enough to contain the circuit board was created in the wood, so that the ultrasonic sensor would be able to stand upright. The luminosity sensor and LED were also connected to the top of the sensor box. Because there were already spaces between the poles at the bottom of the box, more holes did not need to be drilled in order to have the wires for the bottom circuit board to pass outside. A small cap was glued to the bottom of the metal box, so that the entire weight of the box with the sand would balance on the area of the force sensor. This was then placed on the force sensor. As a result, the only thing left to do was to glue all the boards down, to ensure they were secured in place.

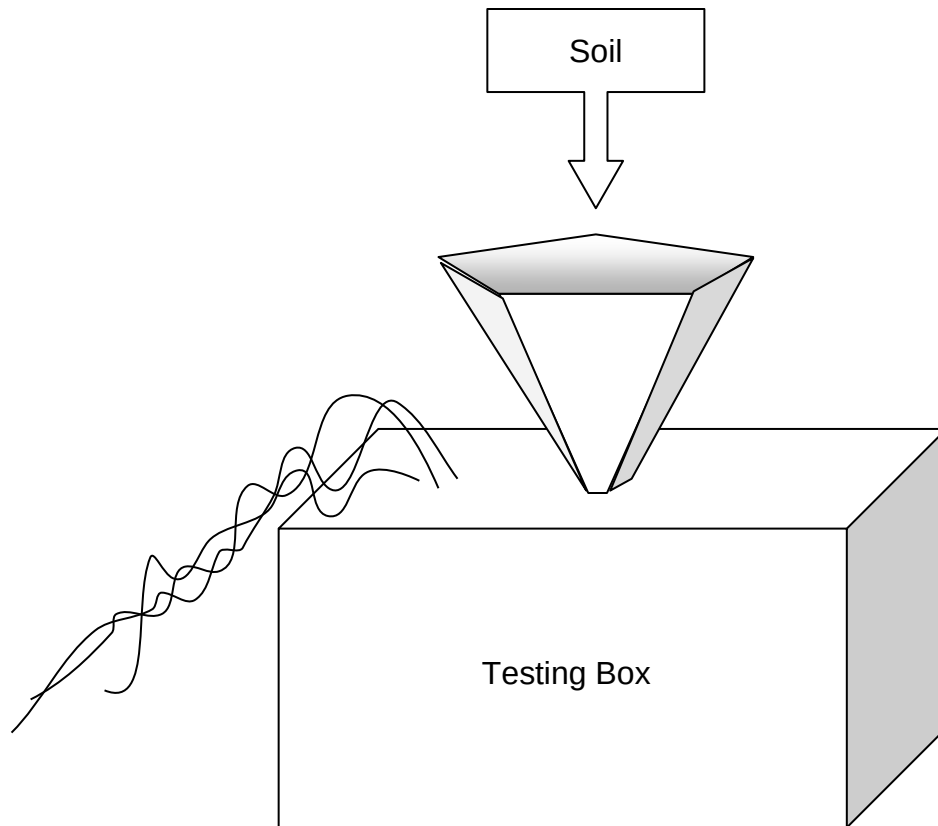


Fig 35. Testing Box. Soil falls through the top of the funnel into the testing box, where the various sensors are. They will measure the properties of the soil. This information will be communicated back to the Raspberry Pi, which will make any further calculations and display the values on the graphical user interface.

January 12, 2019: Troubleshooting

The base servo stopped functioning. Several possibilities and reasons were:

- 1) The base servo was broken: the servo had been working previously.
- 2) High torque servo required different coding requirements: it was noticed that while the high torque servo did not work, the standard servo was able to work with the same code.
- 3) It required more current: the base servo endured the greatest load and possibly required more current to power its movements.
- 4) It required more voltage: by having more voltage drop across the battery, the current would be higher, resulting in greater power
- 5) It required less voltage: upon searching up the properties of the high torque servo, the group found that the servo worked under 4.8V to 6.0V operation. It was currently connected to a 9.0V battery.
- 6) The bottom was creating too much friction: unlikely, but still a possibility

The group tested out the possible errors. Comparing the performance of new servos, the group tested out the code. The servos still did not move. Since they were new servos, the group ruled out possibility one.

Lubricant oil was added to the bottom of the base where the stilts were added. Nothing happened. The rotating plate was a little easier to move, but not significantly enough that it would have a drastic effect on the motor's ability to move. Possibility six was ruled out.

The group considered reasons three and four -- not enough current and power -- for why the base servo wasn't functioning. However, since the battery used was 9.0V, which was already higher than the threshold (4.8V to 6.0V), they decided that it was not the case, and that perhaps it was the reason why the servo was not functioning.

The group decided that number five -- the battery voltage was too high -- was most likely the reason why the servos did not work. The 9.0V battery was replaced with a makeshift 6.0 battery pack and tested again.

This time the base servo pulsed and worked. The group reasoned that the 9.0V battery produced too much potential drop which overloaded the servo, causing it to jam and to not move at all. The group decided to use a battery with voltage drop between the ranges of 4.8V to 6.0V.

A makeshift battery pack was made out of four 1.5V AA batteries as a battery pack was not yet found. The group agreed to buy it the next day. Down below is an image of how the battery packs were made. Four 1.5V AA batteries were stacked together in series to make a 6.0V battery. A multimeter was used to check if a connection occurred. Elastic bands were used to ensure that the battery ends made contact and were conducting a current of electricity.

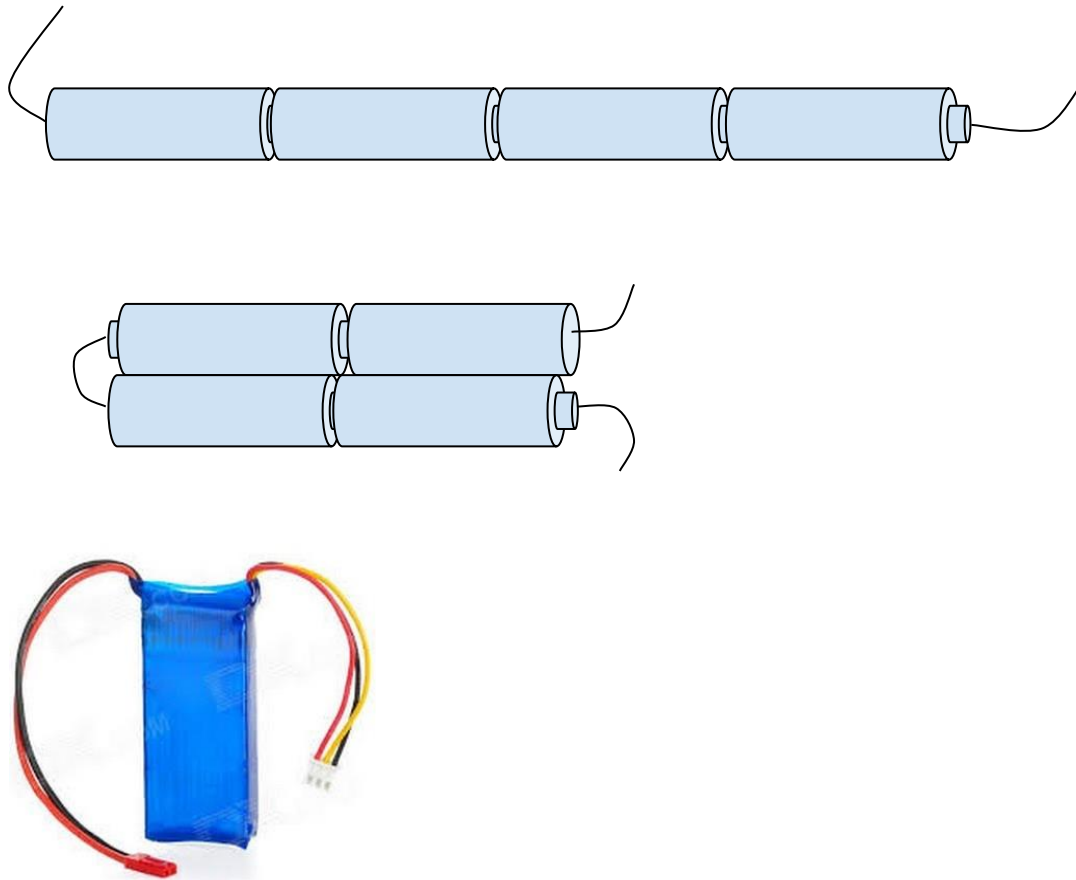


Fig 36. Battery diagram. Four 1.5V batteries were connected in series. To reduce space, the second design was used. The batteries are then connected together and taped together using duct tape.

After realizing that the 9.0 V battery was causing the issues, the group decided to change all the batteries to 6.0V. More makeshift battery packs were made for the time being. Battery packs will be bought the following day.

It was also decided that the funnel opening for the soil testing box was too small. As a result, extensions to the top of the funnel were made with cardboard and tape, so that the arm would have more flexibility for error when dropping the soil in the testing station.

January 13th, 2019: Final Testing

Multiple test runs were conducted to see if the arm could move to the desired position accurately, and deposit the soil it collected into the testing station. A cardboard box similar to the one that would be used on testing day was filled with beads, to mimic sand. It was placed one metre away, at a slight angle. Using the blue ball detection method, the livestream was updated so that the arm would move closer to the desired location. Many of the first few runs were unsuccessful, as the arm would always move close to the desired location, but sadly it would land just outside the box and miss the sand entirely. At times, the movement of different parts of the arm were just skipped entirely. These issues were fixed by better calibrating the angle accuracy of the servo motor in relation to the IP webcam. It was also discovered that if the blue ball leaves the frame of the camera, it is not detected, so the software just constantly updates and the arm lowers continuously, forever. These issues were alleviated by tweaking the software.

Finally, the arm reached the soil box and collected soil. However, on the way up, the claw was not constantly moving with the rest of the arm to ensure that it is always upright, and as a result, some of the soil spilled. This was corrected in the code. This movement was tested multiple times to ensure repeatability and validity. Finally, the soil testing station was placed in the frame of the camera, at a specific location, and this information was inputted into the software. The arm was then programmed to move towards the box after picking up the sand. The sand then dropped down the funnel into the metal box in which all the experiments would be conducted. This process was tested multiple times in order to confirm that it was reliable.

January 14th, 2019: Budget

At the end of the project, the group added up total costs required to create the arm. Receipts were compiled and costs were organized into an excel. The total was \$179.21, which was slightly higher than the predicted budget of \$174.25 in the prospectus.

Item	Price
Testing	
ADC	17.49
Temperature Sensor	5.85
Luminosity Sensor	8.49
Force Sensitive Resistor	9.95
Ultrasonic Sensor	4.99
RGB LED	1.79
Wire	4.99

Micro Servo	10.25
High Torque Servo Motor	23.99
Standard Servo Motor	14.49
Continuous Servo Motor	18.95
6V AA Energizer Batteries	22.99
Raspberry Pi	34.99
Total	179.21

Fig 37. The costs were added together at the end of the project. The total spent was \$179.21.

ZOROAK: OPERATING MANUAL

SET UP, SOFTWARE AND HARDWARE USE

SOFTWARE

Code Installation

On the user's computer with a linux terminal with python2 and python3 installed, the user is required to install the following libraries/packages:

cv2
imutils
numpy
kivy

flask

The user should have the following files in the same folder on their computer:

Webcam.py
bash2.sh
Output.py
ColourReferenceScale.jpg
webserver.py

On the Raspberry Pi with python2 and python3 installed, the user is required to install the following libraries/packages:

requests
gpio
Adafruit_ADS1x15

The user should have the following files in the /home/pi/Amy folder on the Raspberry Pi:

ALLSENSORS.py
ARM_MOVEMENT.py

Network Configuration

All devices should be connected to the same network with the same subnet. This would allow the devices to seamlessly connect with each other.

Change the ip address in the bash code to the proper ip address of the Raspberry Pi.
Change the ip address in the ARM_MOVEMENT.py to the proper ip address of the computer.
Change the ip address of the Webcam.py to the proper ip address of the phone ip webcam.

User Interface

The user interface consists of two separate applications. One of them will display the live camera feed and the image that the user can click on. The other will display the results of the testing for the soil sample.

Starting the Arm

1. Make sure that everything is connected and turned on.
2. Input “bash bash2.sh” onto the terminal command line.
3. The live camera feed will pop-up and after a few seconds a still-image will pop-up on a separate window.

4. Follow the instructions on the still-image screen to click on the place of soil sampling and to start the robot.
5. The other steps will occur autonomously and the final results will be displayed on a separate application to the user.

HARDWARE

Arm Setup

1. Place the arm in a suitable position around 1m away from the sample.
2. Ensure all the arm components are in their original positions.

Testing Station Setup

1. Place the testing station 0.8 m from the sample station.
2. Make sure all the wires are connected properly to the raspberry pi.

Camera Setup

1. Open the ip webcam application on the phone.
2. Place the phone on the stand.

Using the Soil Analyzer

1. Place the soil analyzer on a stable surface/ground
2. Choose a location at least 1m away to test soil
3. Select this location on the image on the screen of the computer
4. The computer will display the properties of the soil after approximately 30 minutes

ZOROAK: DEVIATIONS FROM PROSPECTUS

CHANGES IN HARDWARE, SOFTWARE, AND DESIGNS

There were a few minor deviations in the final design of the remote soil analyzer compared to the prospectus. These changes are categorized into the following groups: deviations in materials, software, and the testing station.

Materials

The arm was initially to be built from 3x3x12 inch blocks of balsa wood. These wood blocks were bought, but then a fishing rod was found unexpectedly. Because of the sturdy yet light nature of the rod, the circular rings connected along the arm, and the lack of payment needed to use the fishing rod, the balsa wood was returned. If the fishing rod hadn't been free, it likely wouldn't have been bought and use. As a result, it was difficult to anticipate if we would find the broken fishing rod.

Originally, the continuous servo motor that pulls the arm up and down was to be situated directly on the rotating base of the arm. However, after doing this, the force exerted from the arm to the servo was so great that it looked like the servo would break. As a result, pillars were added to elevate the servo so that there would be a greater mechanical advantage when pulling. This was not anticipated because the servos were not purchased beforehand, and the force that they could withstand was unknown.

After constructing the arm, the base was very wobbly because the entire rotating part was balanced over a tiny area. Pillars were added underneath the base to stabilize it. This was not anticipated because the area of the servo motor connection was unknown, as it was not obtained by the time the prospectus was delivered. As well, the new weight of the fishing rod was unknown, and this contributed to the instability.

Software

There was only one main change to the coding system planned. Originally, the group intended to use an Arduino as the only microcontroller. After writing code for some of the sensors, it was realized that it was highly inefficient due to having to re-download the code every time in order to edit it. For this reason, a Raspberry Pi was used instead. This was not anticipated because the group did not research this aspect.

A blue ball was also added to the arm. This is to be used as a target on the depth sensor, so the sensor can determine the height of the arm from the ground. Originally, an ultrasonic sensor was to be connected to the claw, to determine how low the claw needed to move, however, it wasn't as accurate at larger distances (over 50 cm).

Testing Station

Originally a photocell/photoresistor was included in the prospectus for the reflective spectra. The photocell was not bought for the purposes of this project; a group member had it beforehand. When testing it, it was realized that the resistance produced needed to be graphed against luminosity. Originally, the group wished to find a graph of this online using the model number, but it could not be found. As a result, a luminosity sensor was used. This was not anticipated beforehand because it was assumed the graph could be found online.

Originally, a cuvette was to be used to hold the soil for all the testing. However, upon visiting a store, it was sold out and could not be found in a convenient Toronto location. Ordering online only comes in bulk at prices in the hundreds. As a result, a larger box was used to keep the soil.

Additionally, no rotating base inside the testing station was added. This was because it was deemed unnecessary. It was not anticipated beforehand because the group members over-thought this aspect.